



TECHNISCHE
UNIVERSITÄT
DARMSTADT

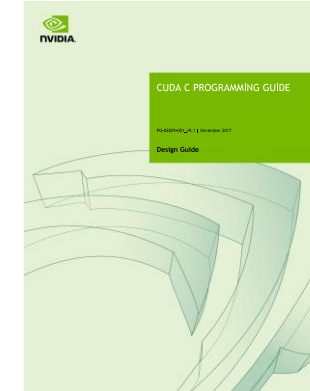
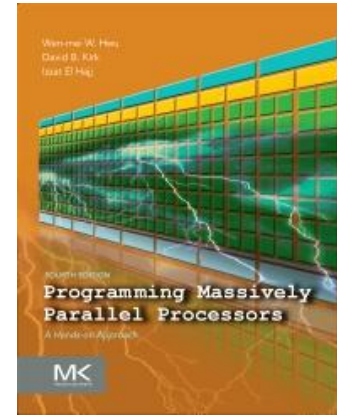
Parallel
Programming

Parallel Programming - CUDA -

Prof. Dr. Felix Wolf
Technical University of Darmstadt

Literature

- Programming Massively Parallel Processors
David B. Kirk, Wen-mei W. Hwu,
4th edition, Morgan Kaufmann, 2022
- NVIDIA CUDA C Programming Guide
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- D. Patterson and J. Hennessy: Computer Organization and Design,
The Hardware / Software Interface, 5th edition, 2014
- J. Meinke, J. Kreutz, W. Homberg, D. Becker, S. Rinke, T. Lanfear, G. Ziegler:
GPU Programming Course, Jülich Supercomputing Centre, Germany,
October 4–6, 2010



Outline

- Evolution of GPUs
- GPU architecture
- CUDA execution model
- CUDA programming interface
- Thread organization
- Exploiting device memories
- Advanced GPU features

Graphics Processing Unit (GPU)

- Generates 2D and 3D graphics, images, and videos
- Applications in graphical user interfaces, video games, visual imaging applications
- Modern GPU – highly parallel, highly multithreaded multiprocessor optimized for **visual computing**
 - Real-time visual interaction with computed objects
 - Unified graphics and computing architecture as a synthesis of
 - Programmable graphics processor
 - Scalable parallel computing platform

Brief history of GPUs

1980s – video graphics array (VGA) controller

- Memory controller and display generator connected to some DRAM

Late 1990s – 3D accelerator functions added to VGA controllers including hardware for

- Triangle setup and rasterization (dicing triangles into individual pixels)
- Texture mapping and shading (applying patterns to pixels and blending colors)

2000 – almost equivalent to high-end workstation graphics pipeline

- Term GPU was coined to denote that the graphics device had become a processor

Brief history of GPUs (2)

Over time, GPUs became

- **more programmable** – fixed function logic replaced by programmable processors
- **more precise** – from integer to fixed point, to single precision, to double precision
- **more scalable** – disparate programmable pipeline elements merged into unified array of many programmable processors. Improved load balancing

Support for **general-purpose programming languages** such as C and C++ was added – made GPUs fully programmable

Now also geared towards **AI workloads** (mixed-precision tensor cores)

Major application for GPUs – Deep learning

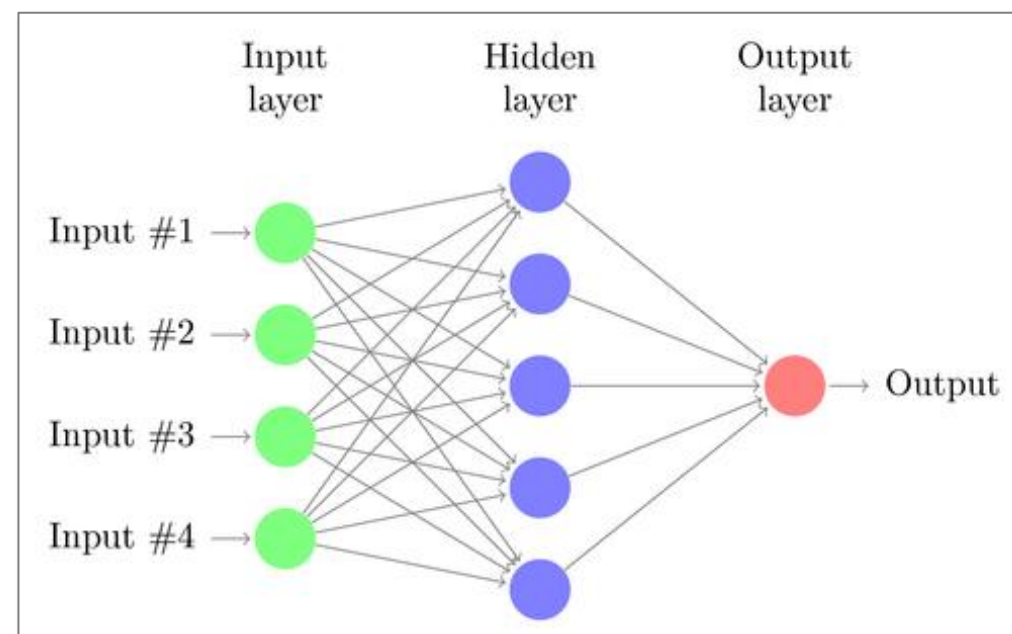


TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming



Yes or
no?



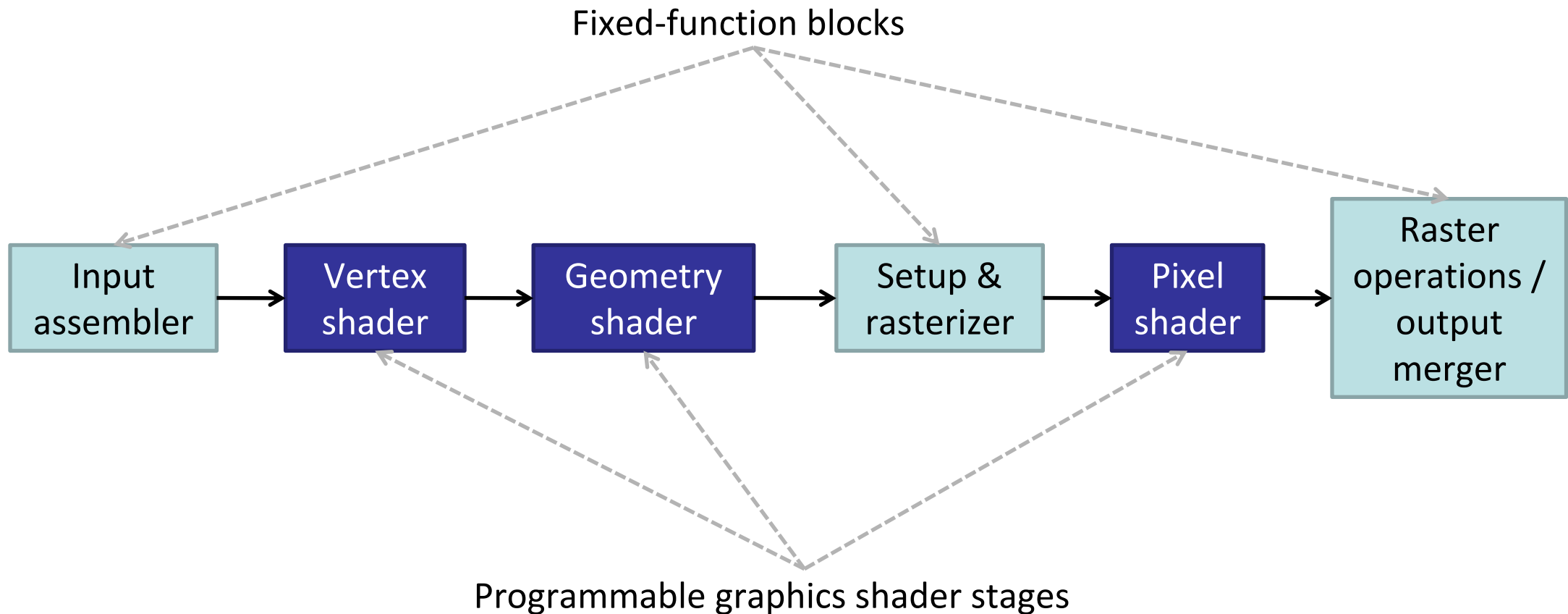
Neural network

Logical graphics pipeline



TECHNISCHE
UNIVERSITÄT
DARMSTADT

 Parallel
 Programming

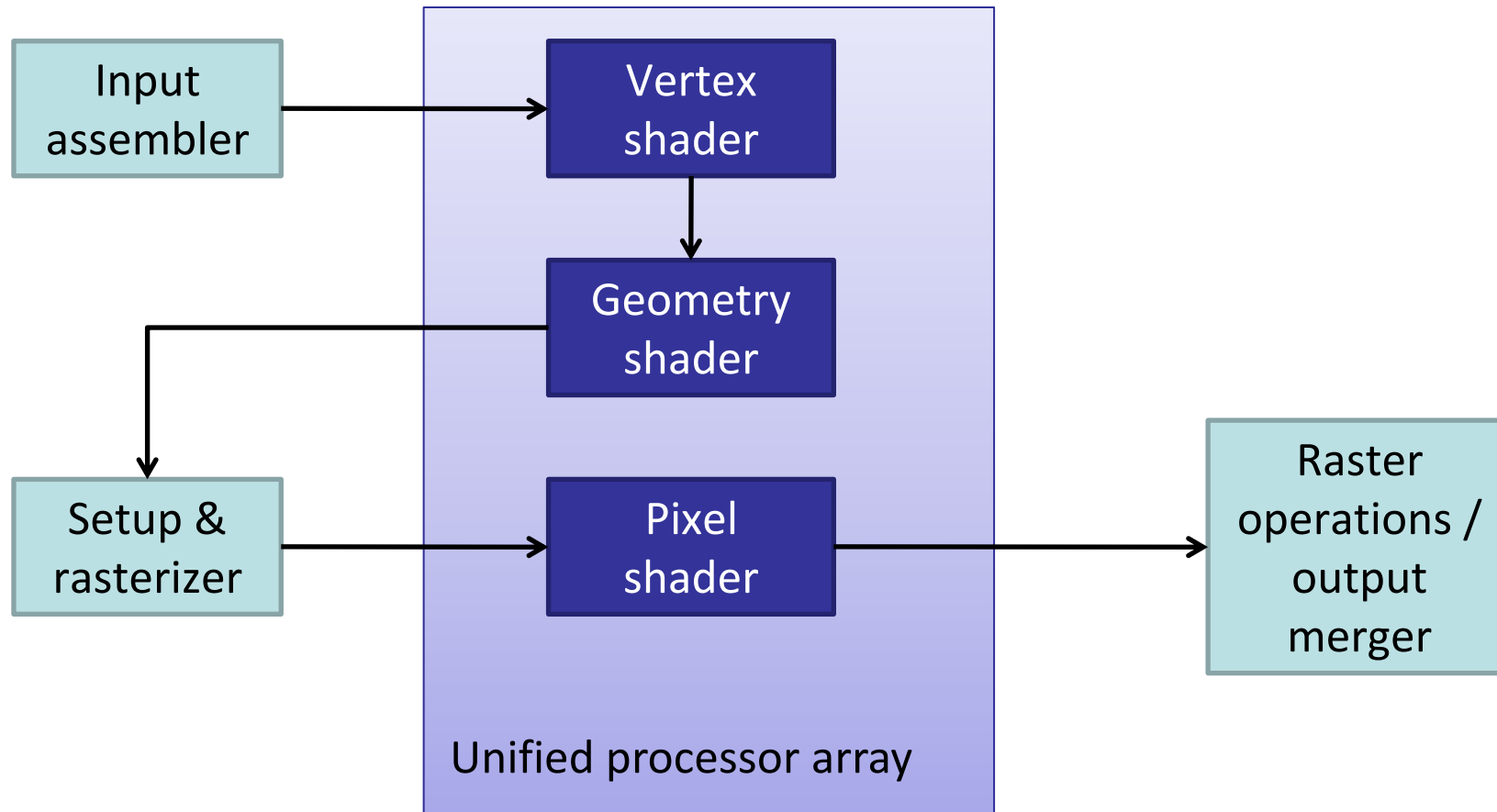


Mapped to physical processors



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

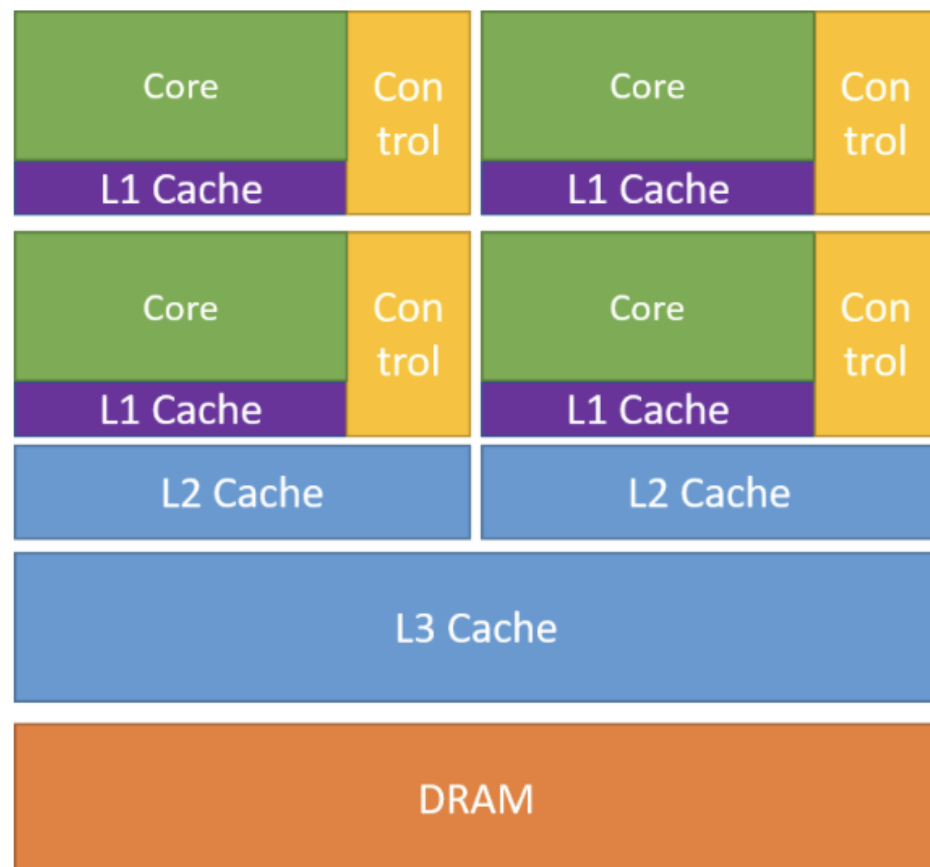


GPU – more transistors for data processing

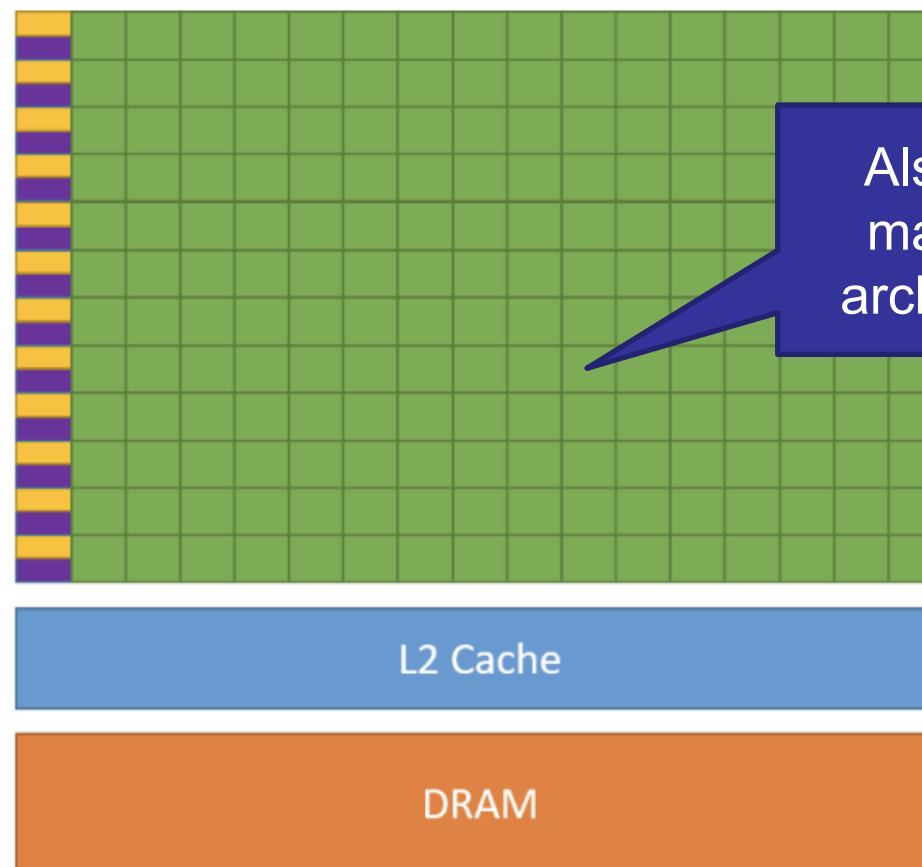


TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming



CPU



GPU

Also called
many-core
architectures

GPUs designed for data parallelism

Functional parallelism



Construction workers

Data parallelism



Hollow square formation

CPU vs. GPU

CPU

- Smaller number of cores: $O(100)$
- Complex cores with control logic to maximize instruction-level parallelism
 - Out-of-order execution
 - Branch prediction
- Relatively large amount of main memory per core
- Efficient exploitation of caches

GPU

- Large number of cores: $O(1000)$
- Little control logic
- Massive number of threads
- Data parallel computations
- Code with high computational intensity (ratio of arithmetic vs. memory access operations)
 - Helps hide memory latency
- Power efficient
- Smaller caches

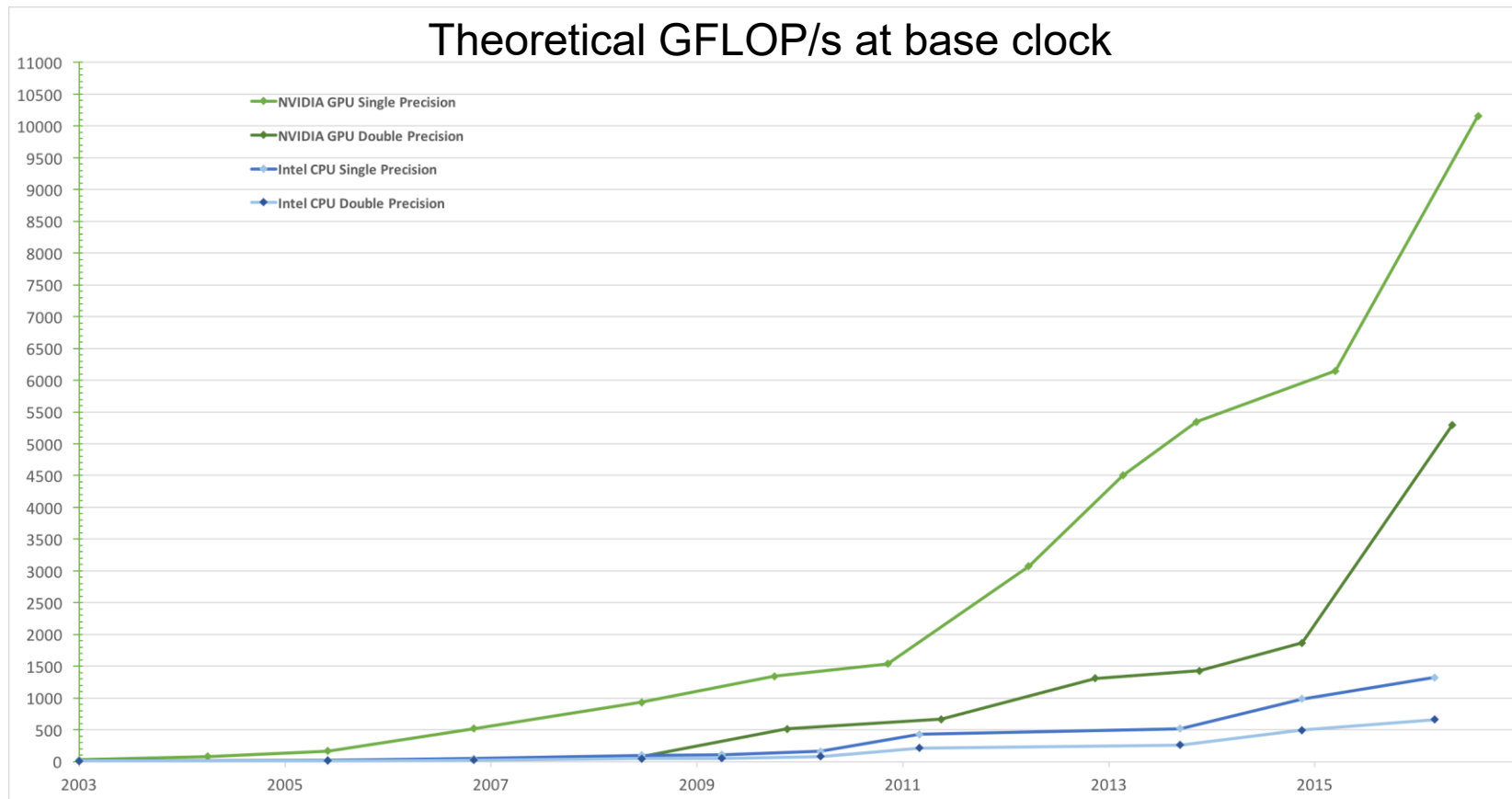
CPU vs. GPU

Floating-point operations per second



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming



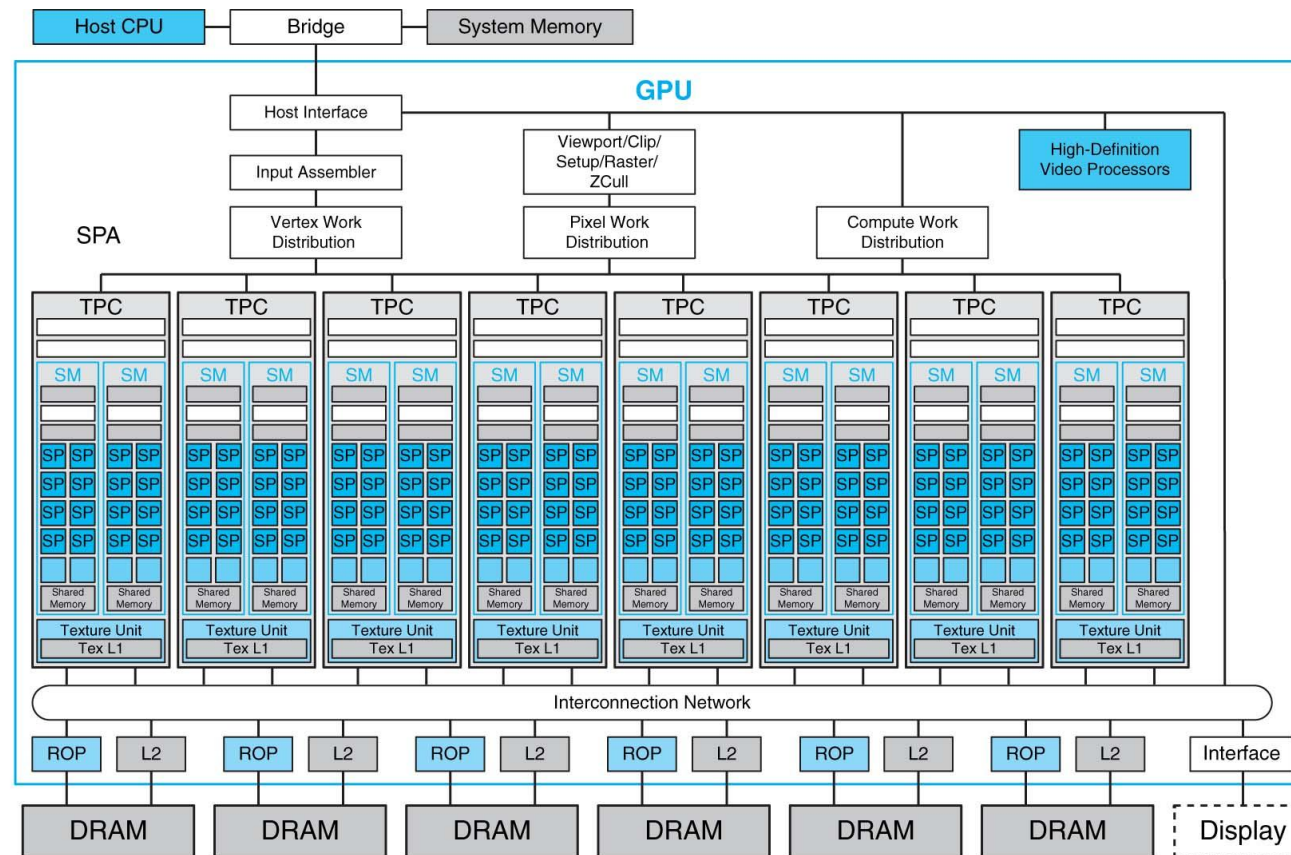
Source: NVIDIA CUDA C Programming Guide V9.1

NVIDIA GeForce 8800 (introduced 2006)



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming



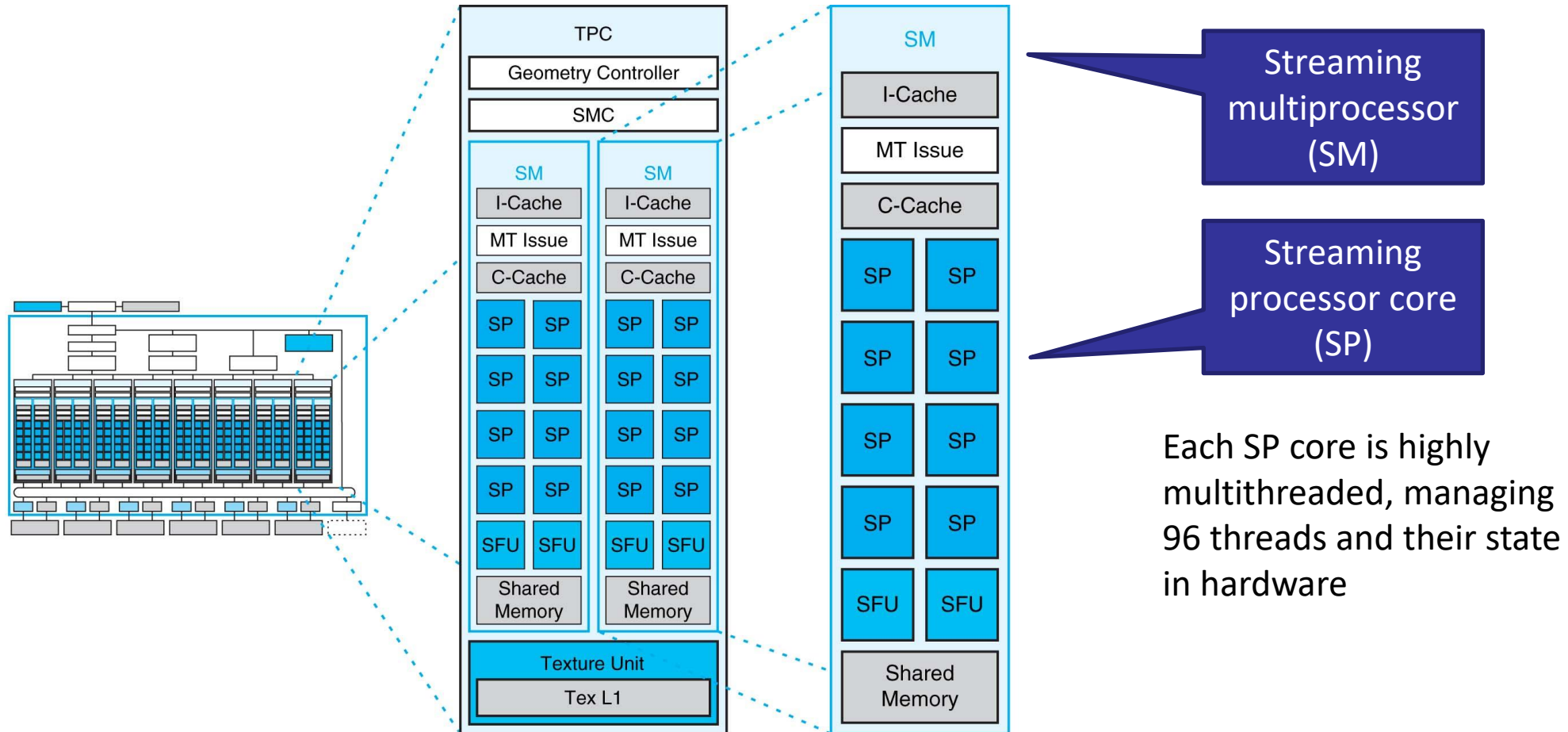
Source: Patterson, Hennessy: Computer Organization & Design, 4th edition, Morgan Kaufmann

NVIDIA GeForce 8800 (2)



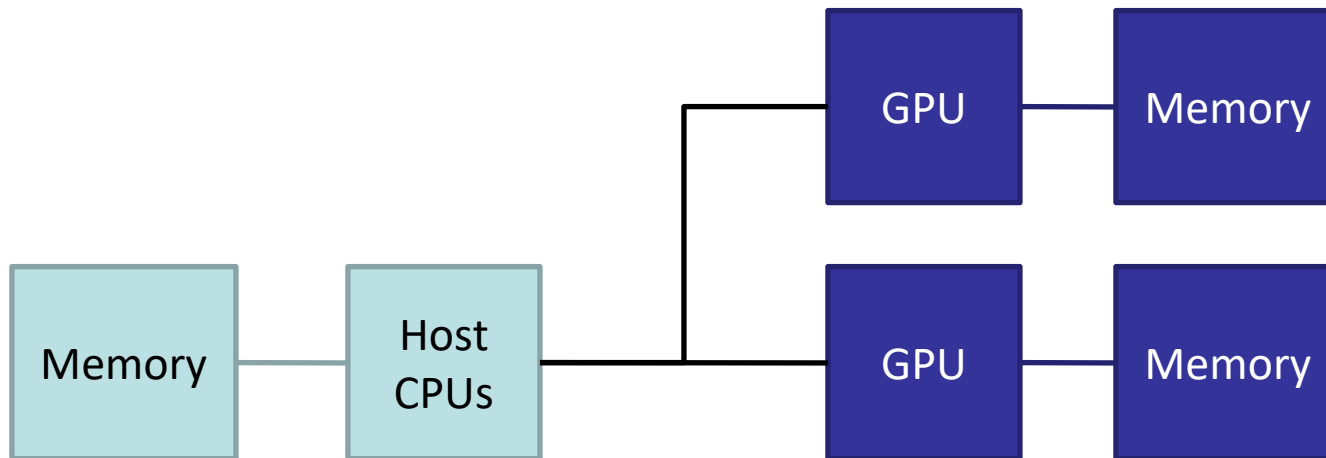
TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming



Heterogeneous system

One or more GPUs form **heterogeneous system** with general-purpose CPU node

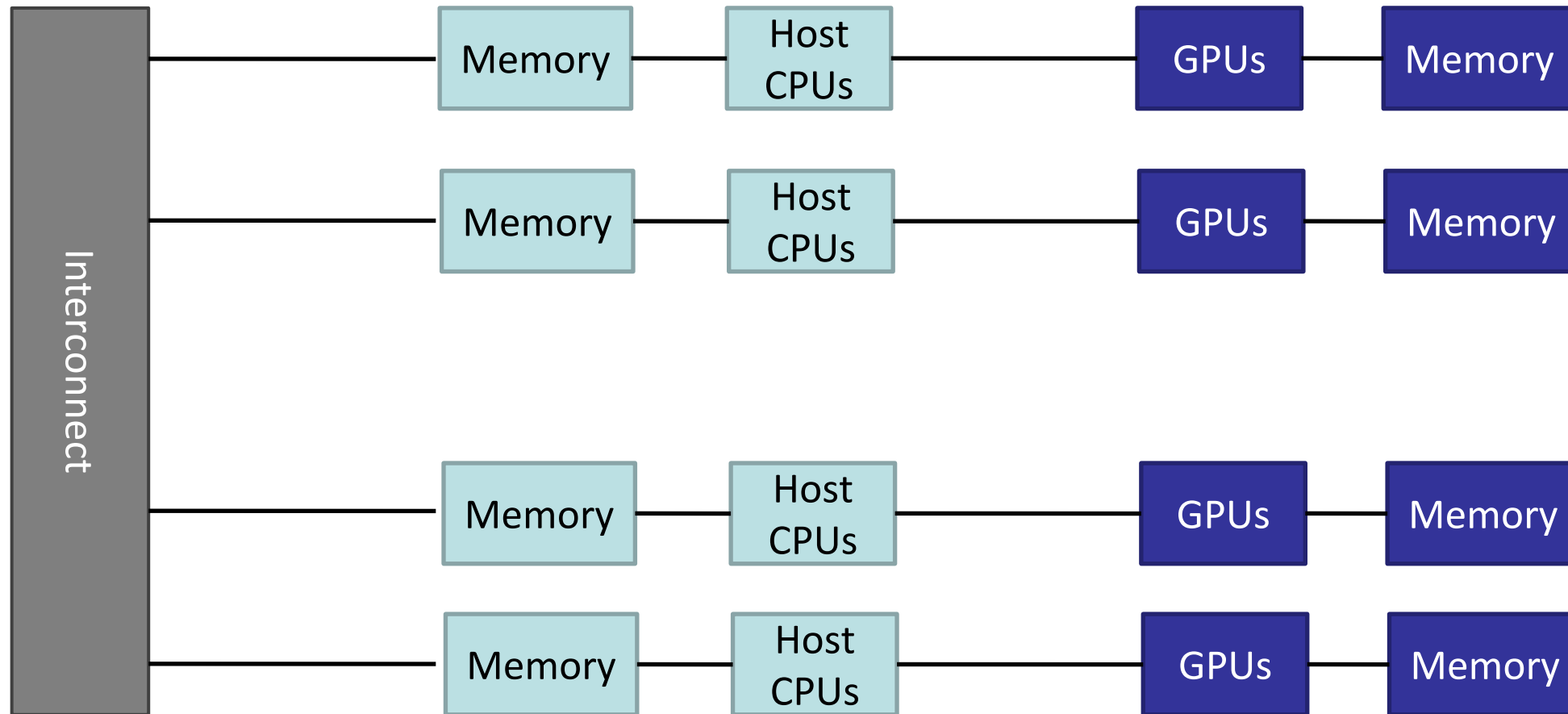


Heterogeneous cluster



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming



GPU programming interfaces



Graphics	General purpose computation on graphics processing unit (GPGPU)
<u>OpenGL</u> – Cross-platform API for writing applications that produce 2D and 3D computer graphics	<u>CUDA</u> (Compute Unified Device Architecture) – extension of C/C++ from NVIDIA
<u>Direct3D</u> – Microsoft DirectX multimedia interface	<u>SYCL</u> – C++ framework for writing programs that execute across heterogeneous platforms consisting of CPUs, GPUs, and other processors
	<u>HIP</u> – extension of C++, targeting NVIDIA GPUs, AMD GPUs and x86 CPUs. Lower level than SYCL.
	<u>OpenCL</u> – C framework for writing programs that execute across heterogeneous platforms consisting of CPUs, GPUs, and other processors
	<u>OpenACC</u> and <u>OpenMP</u> – directive-based interfaces
<u>Vulkan</u> – low-overhead, cross-platform 3D graphics and computing API	

So far...

- GPUs are no longer only used for graphics → accelerator cards
 - Now also tailored to AI workloads
- GPUs differ from CPUs in
 - Hardware design
 - Compute power
 - Application areas
- We use CUDA (Compute Unified Device Architecture) for GPGPU (General Purpose Computation on Graphics Processing Unit)
- <https://docs.nvidia.com/cuda/>

Compute Unified Device Architecture (CUDA)

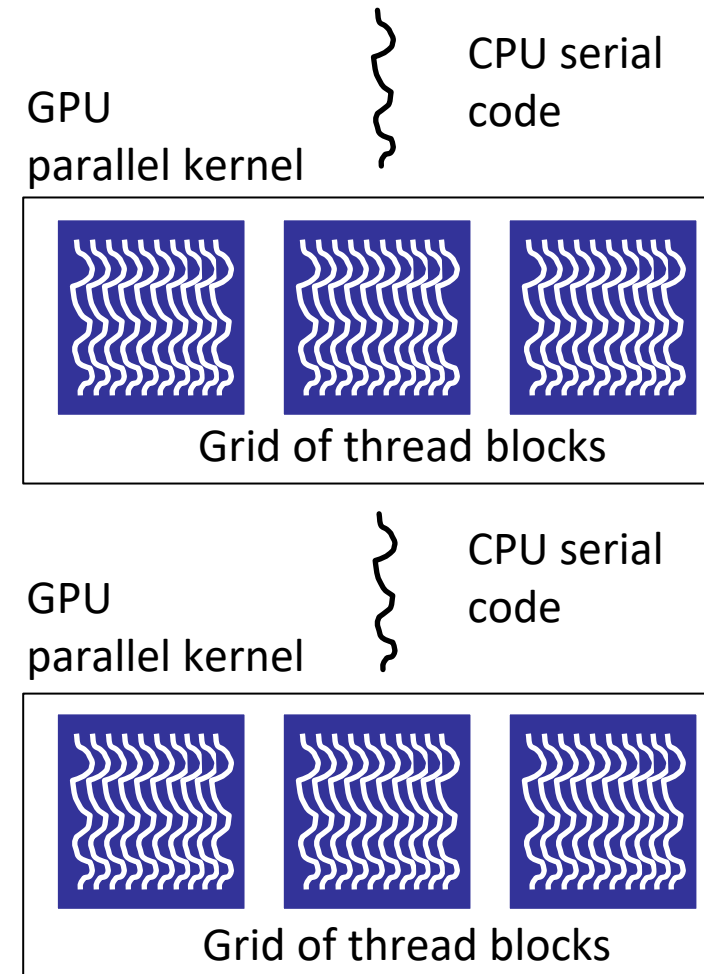
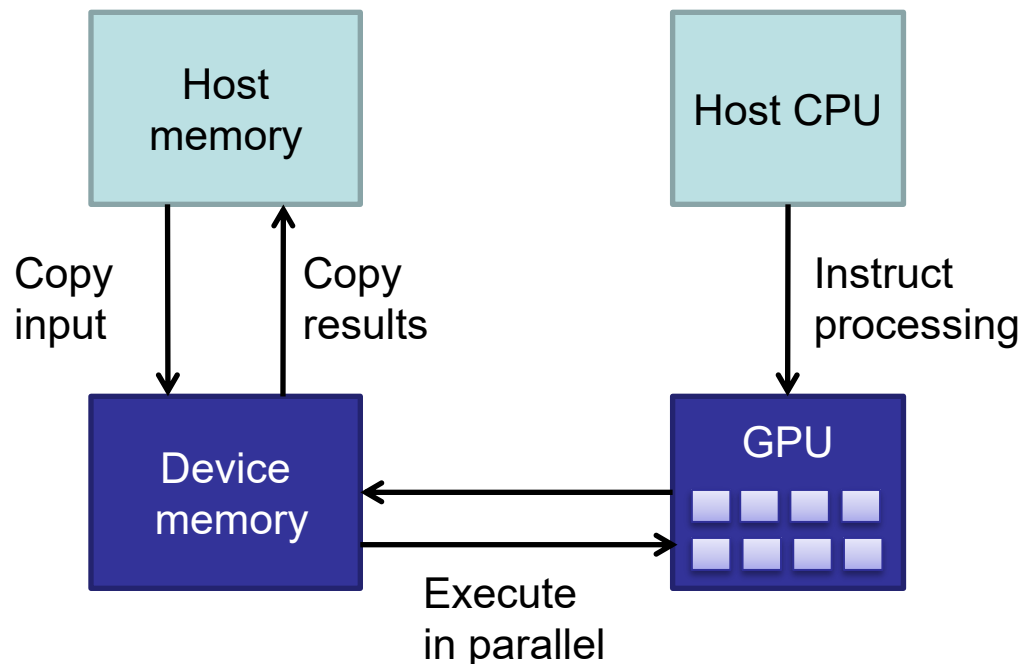


TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

NVIDIA's parallel computing architecture

- C with NVIDIA extensions
- Most recent version 13.0



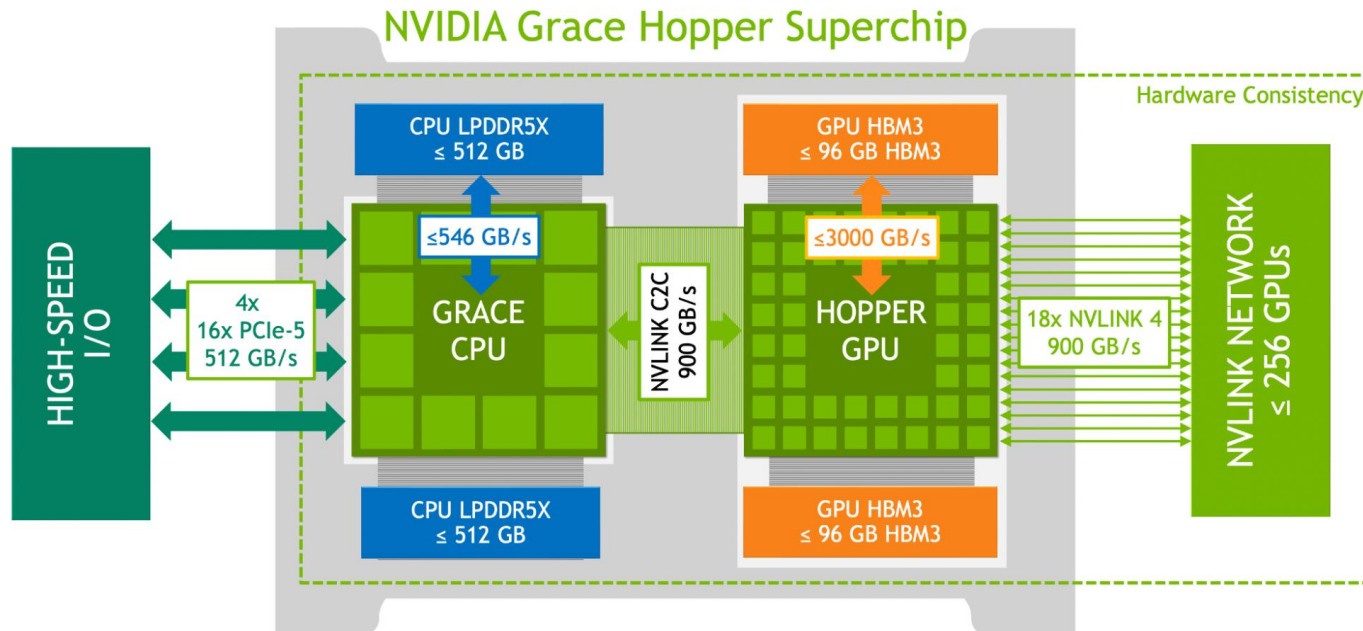
NVIDIA Grace Hopper Superchip



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

- CPU and GPU threads can now concurrently and transparently access both CPU- and GPU-resident memory



CUDA (2)

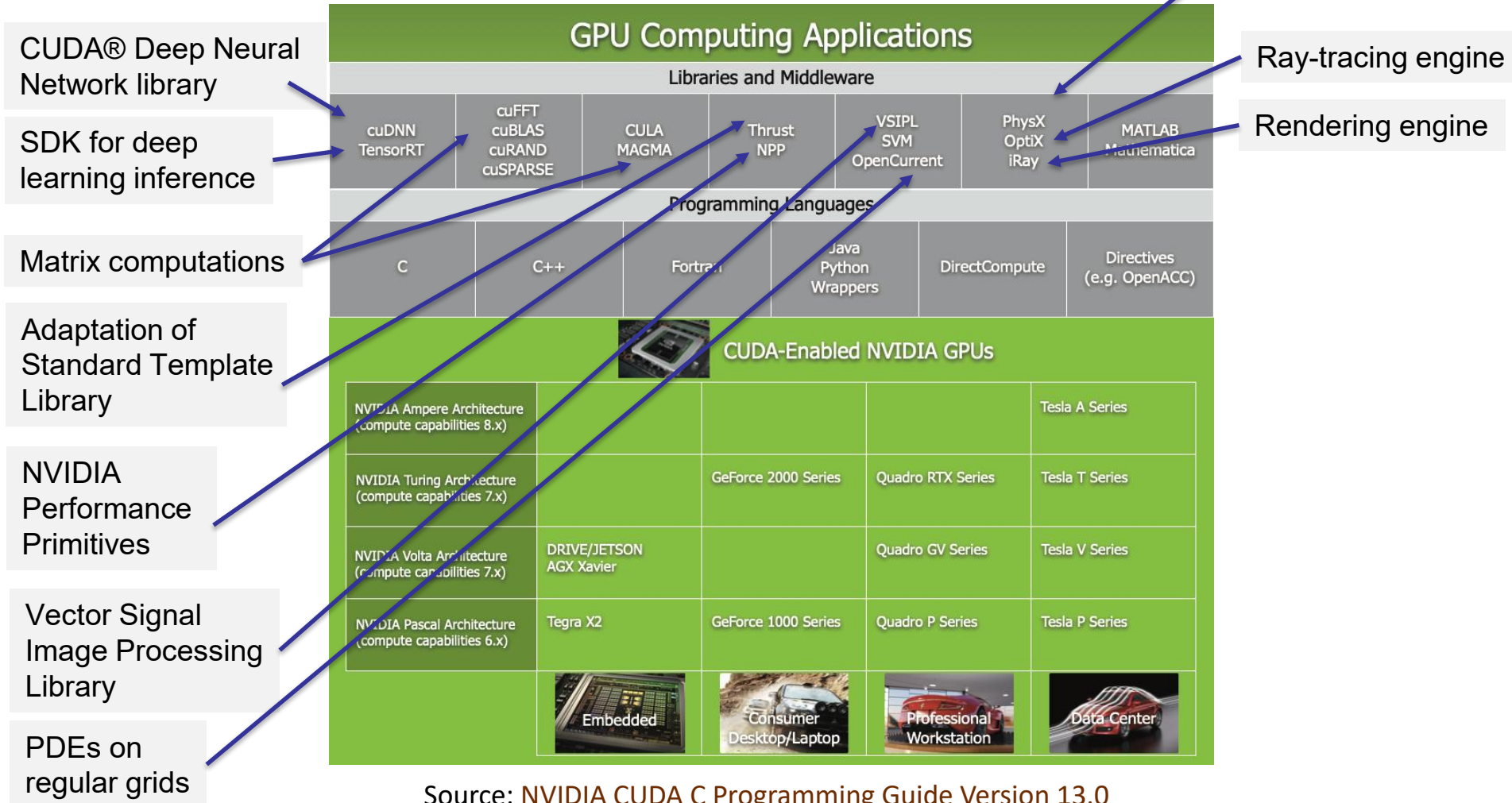
- The GPU is viewed as a compute **device** that
 - is a coprocessor to the CPU or **host**
 - has its own DRAM (**device memory**)
 - runs many **threads in parallel**
- Data-parallel portions of an application are executed on the device as **kernels** running in parallel on many threads
- Differences between GPU and CPU threads:
 - GPU threads are extremely lightweight (very little creation overhead)
 - GPU need 1000s of threads for full efficiency (multi-core CPU needs far fewer)

CUDA software stack



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming



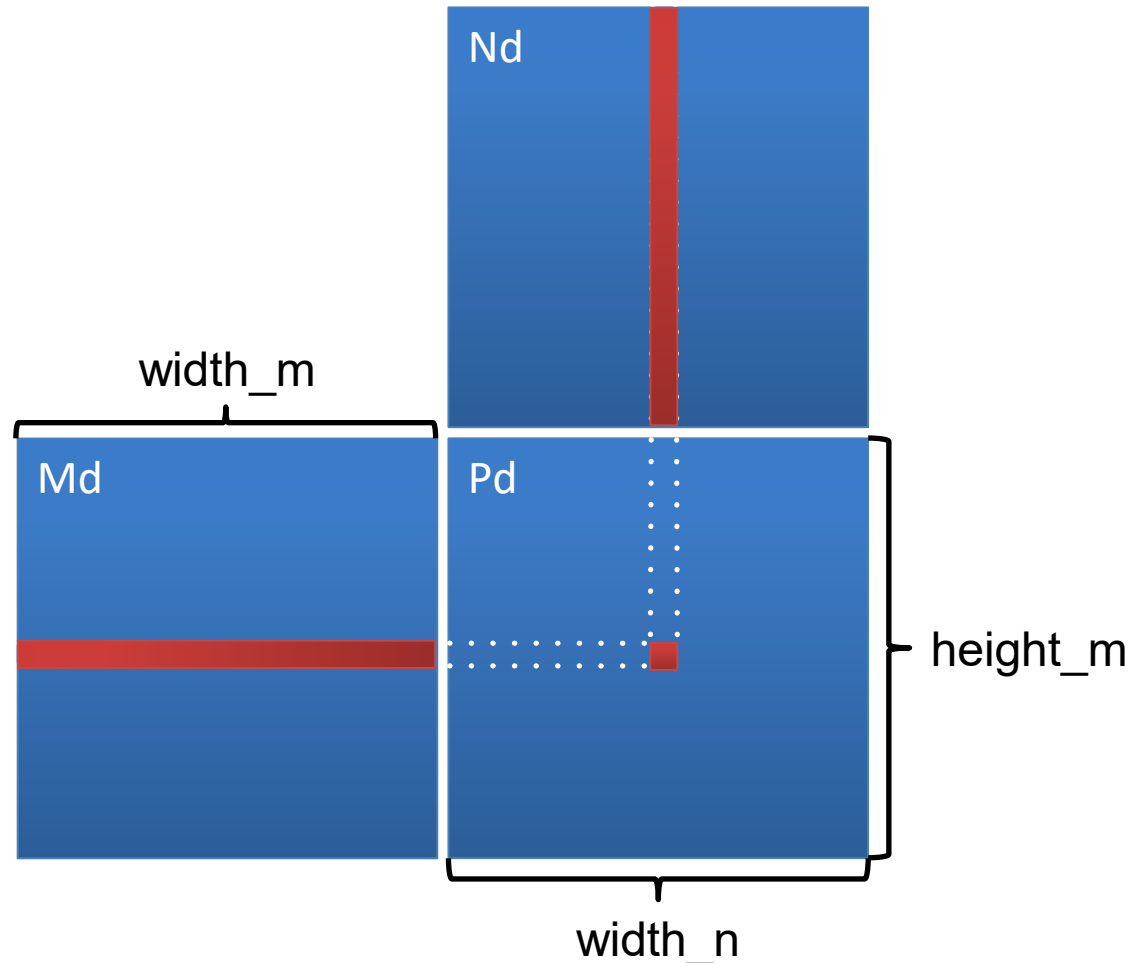
Compute capability

Defined by a major and a minor revision number

- Same major revision number = same core architecture
- Minor revision number – incremental improvement to the core architecture, possibly including new features

Major revision number	Microarchitecture
9	Ada Lovelace / Hopper
8	Ampere
7	Volta / Turing
6	Pascal
5	Maxwell
3	Kepler
2	Fermi
1	Tesla

Example – matrix multiplication

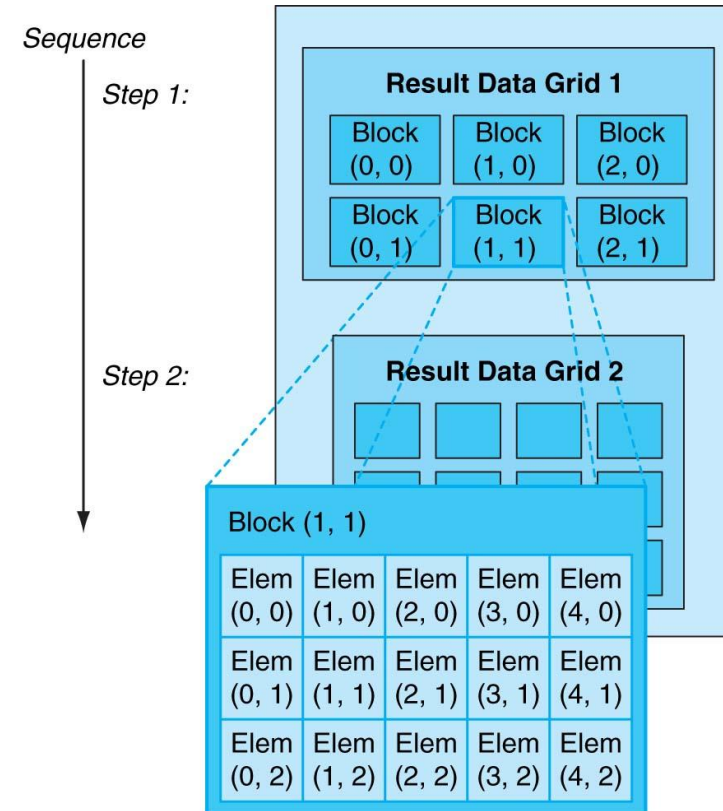


- $width_m * height_m * width_n * 2$ floating-point operations
- Highly parallel, as each output element is computed independently
- Each thread executes exactly the same operations to compute the result

Data-parallel problem decomposition



- **Kernel** computes one grid
- **Grid** partitioned into blocks
 - Blocks can be computed in parallel
 - Mapped onto multiprocessors
- **Blocks** partitioned into elements
 - Elements within each block can be computed in parallel
 - Mapped onto threads
- Each thread computes one or more result elements



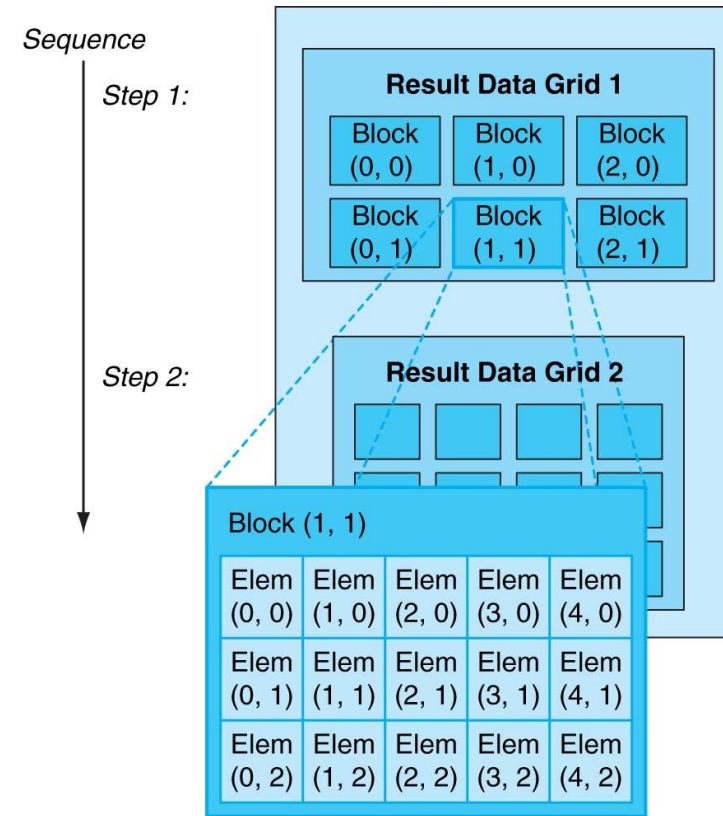
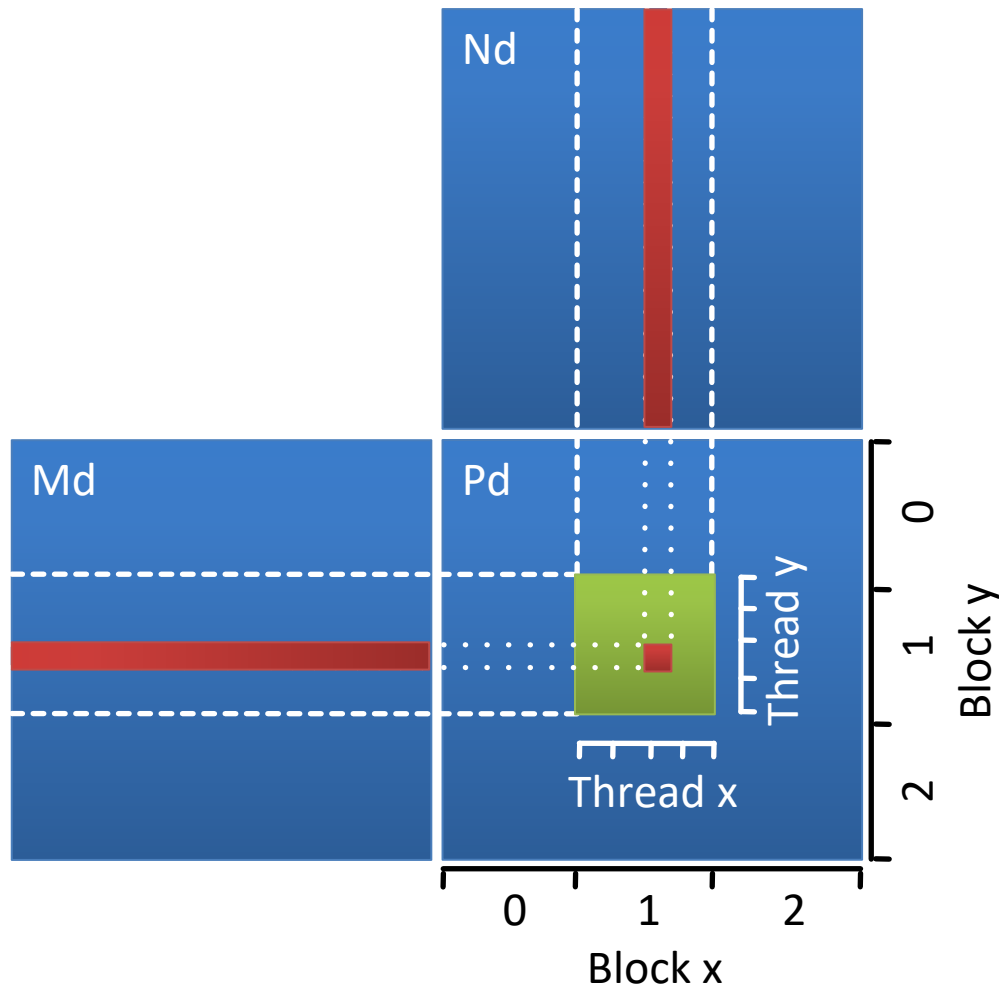
Source: Patterson, Hennessy: Computer Organization & Design, 4th edition, Morgan Kaufmann

Applied to matrix multiplication



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

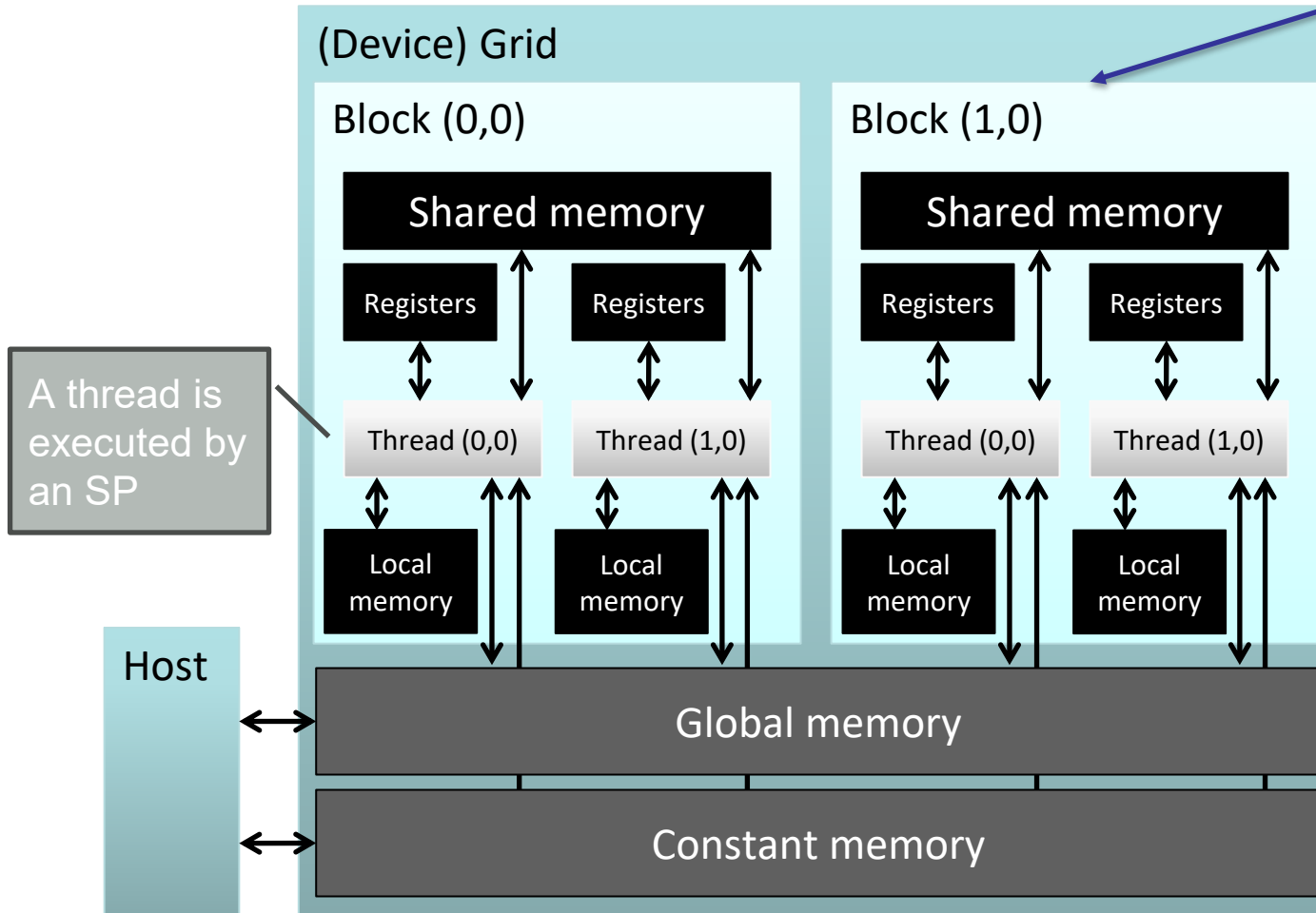


Source: Patterson, Hennessy: Computer Organization & Design, 4th edition, Morgan Kaufmann

CUDA device memories



A block is
executed
by an SM



Each thread can:

- R/W per-thread **registers**
- R/W per-thread **local memory**
- R/W per-block **shared memory**
- R/W per-grid **global memory**
- Read only per-grid **constant memory**

The host can

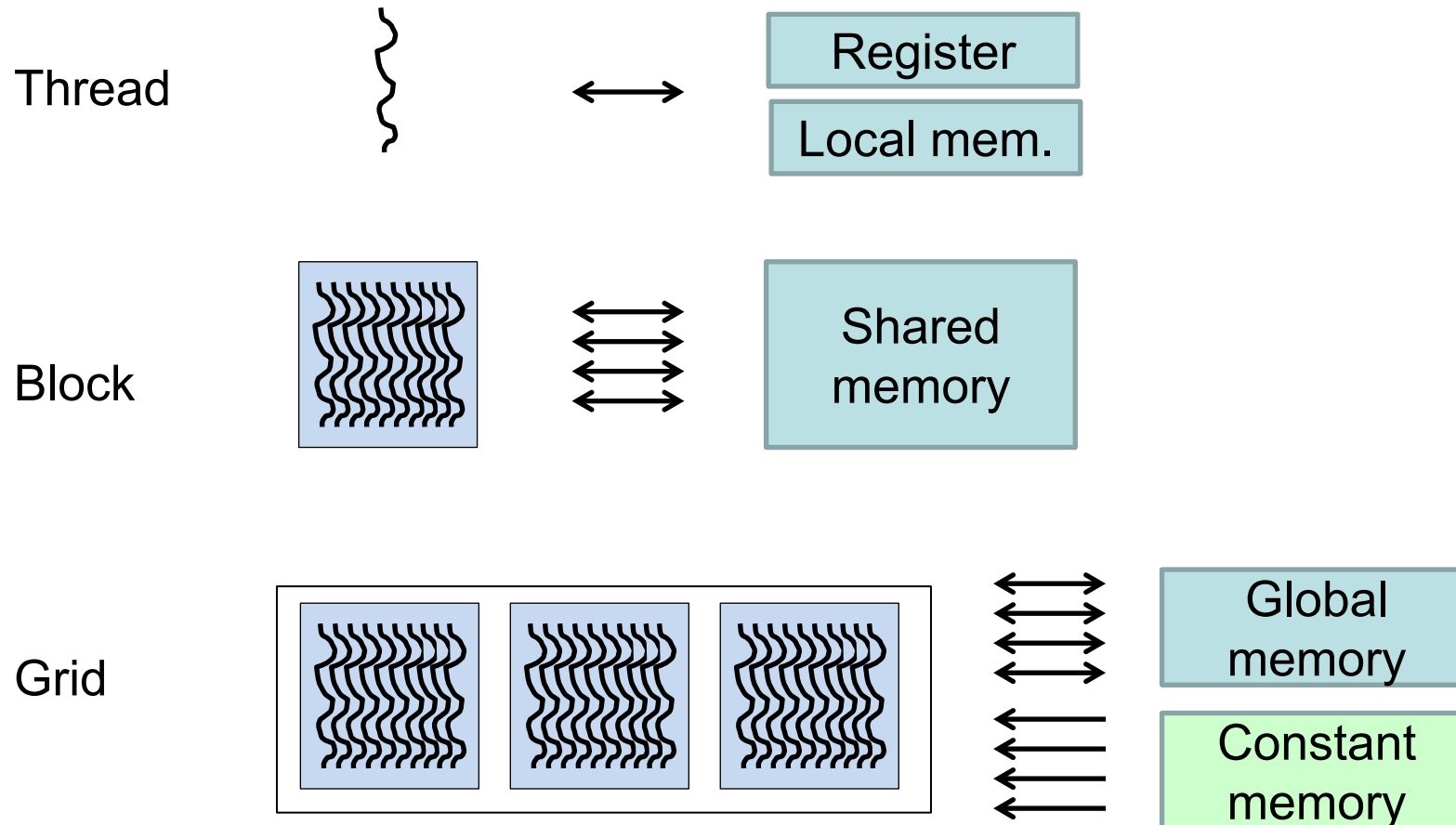
- R/W **global** memory
- R/W **constant** memory

Memory sharing levels



TECHNISCHE
UNIVERSITÄT
DARMSTADT

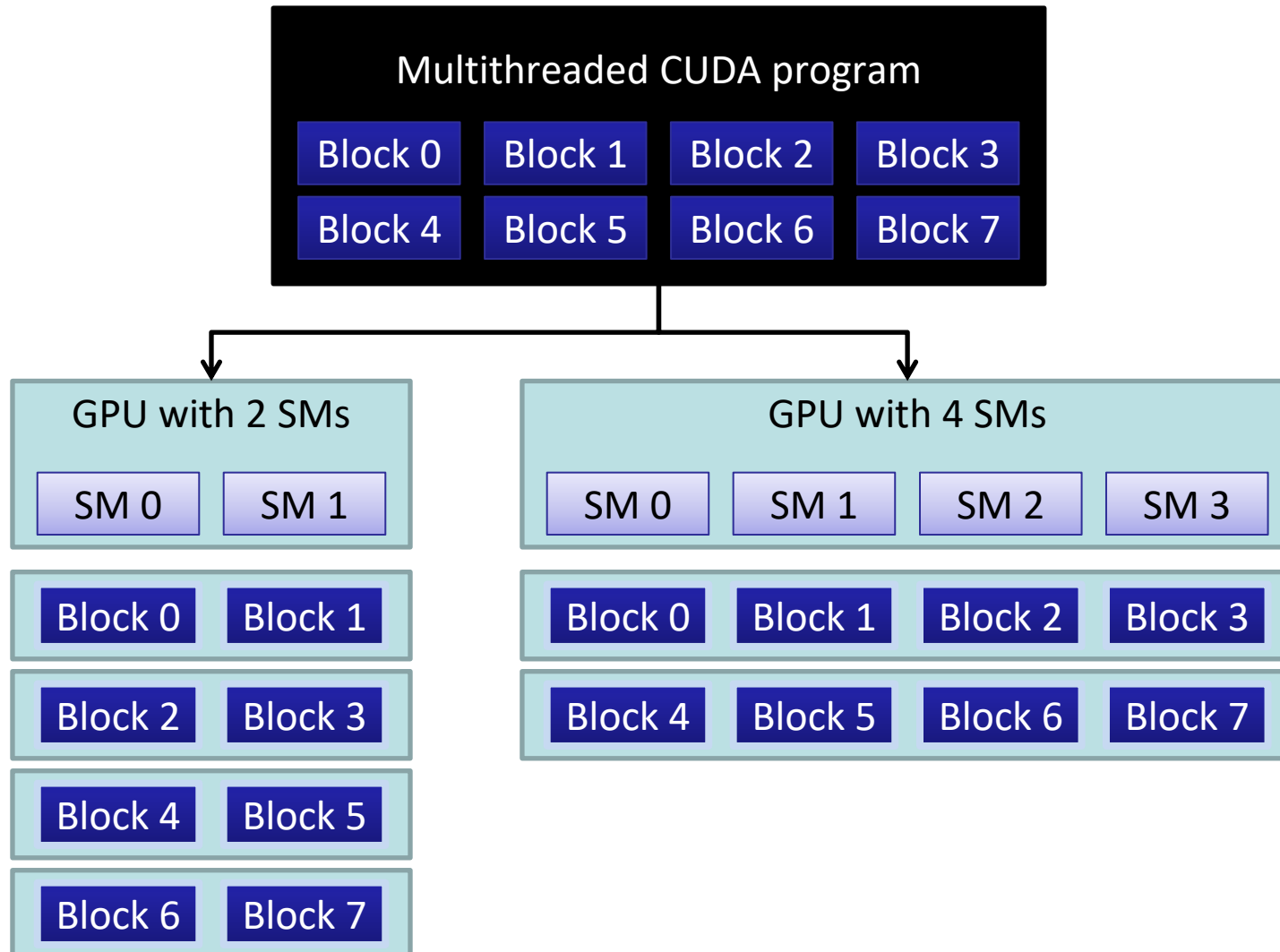
Parallel
Programming



Virtualization

- Threads in a block reside on the same multiprocessor
- Number of thread blocks can greatly exceed the number of multiprocessors
- CUDA virtualizes multiprocessors
 - Granularity of parallelization chosen by the programmer
 - Allows for intuitive problem decomposition: number of blocks can be dictated by the size of the data rather than by the number of processors
- Allows the same CUDA program to scale to widely varying numbers of processor cores

Scalable programming model



Scheduling

- Blocks must be able to execute independently
 - It must be possible to execute blocks in any order
 - Different blocks have no means of direct communication
- However, they may coordinate their activity using atomic memory operations
 - Read, modify, write without intervening access
- Grids may be executed either dependently or independently
 - Independent grids may execute concurrently given sufficient hardware resources
 - Dependent grids execute sequentially with an implicit inter-kernel barrier between them

Parallel execution

- Parallel execution and thread management automatic
 - Thread creation, scheduling, and termination is handled for the programmer by the underlying system
- GPU architecture performs all thread management directly in hardware
- Threads of a block execute concurrently and may synchronize at barrier
 - After passing the barrier, threads are guaranteed to see all writes to memory performed before the barrier (memory barrier implied)
 - Enables communication via shared memory across barrier

So far...

Abstraction	Description
Kernel	• Called from within a serial main program • Simple function or full program • Executes in parallel across a set of parallel threads
Grid	• Set of thread blocks that may each be executed independently and thus may run in parallel
Thread block	• Threads organized in hierarchy of blocks • Set of threads that cooperate among themselves through barrier synchronization and access to memory space private to block • Maximum of 1024 threads per block on current GPUs

So far...



Abstraction	Description
Global memory	• R/W by host and device • R/W per grid • High latency, high bandwidth • Large • This is also where “local memory” is mapped to
Shared memory	• R/W by device • R/W per block • Low latency
Registers	• As registers on CPU • Access per thread • Access in one cycle • Small

CUDA parallelism

CUDA follows single program multiple data (SPMD) model

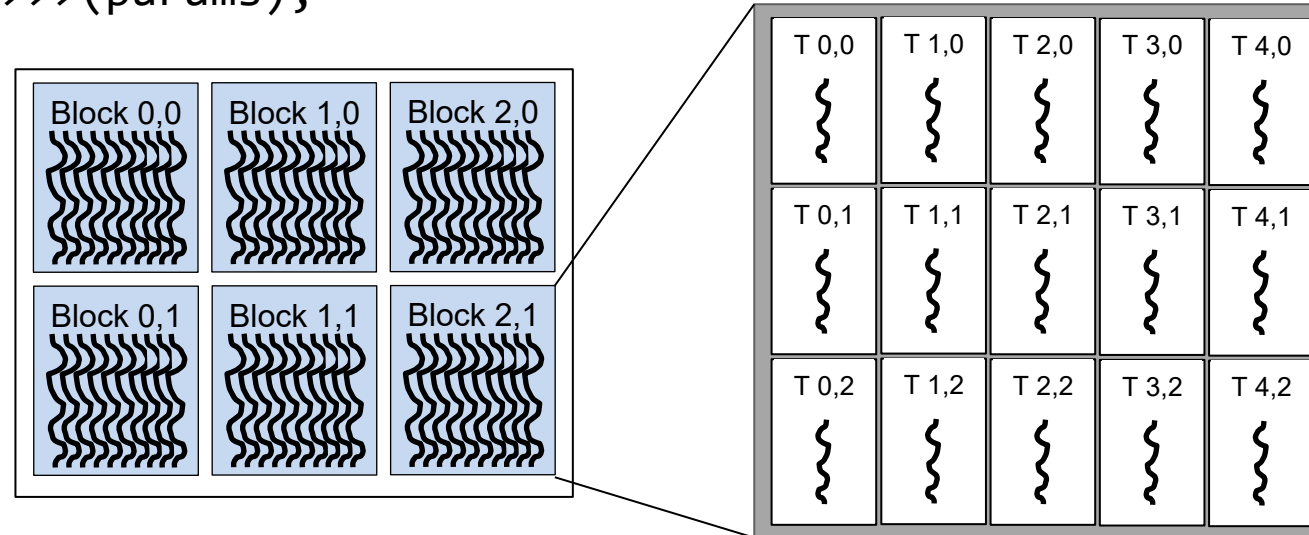
- Expresses parallelism explicitly
- Each kernel executes on fixed number of threads

CUDA runtime entity	Type of parallelism
Concurrent threads of a block	Fine-grained data parallelism
Independent blocks of a grid	Coarse-grained data parallelism
Independent grids	Coarse-grained task parallelism

CUDA parallelism (2)

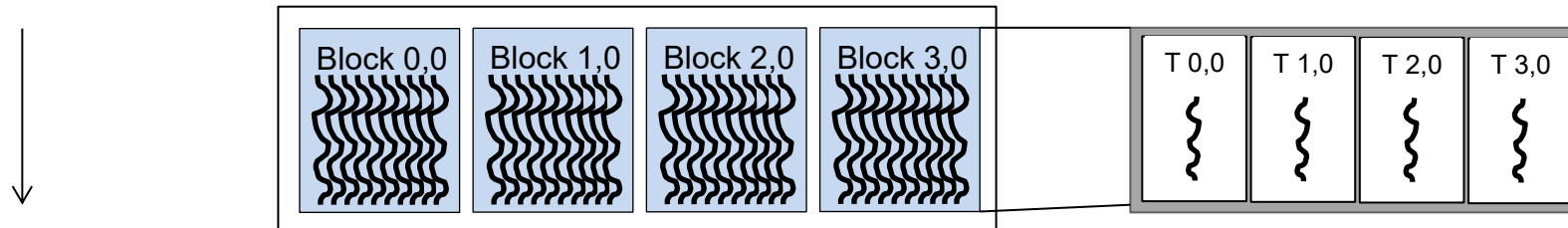


```
kernelF<<<{3,2},{5,3}>>>(params);
```



Inter-kernel synchronization barrier

```
kernelG<<<4, 4>>>(params);
```



Restrictions

- Task parallelism difficult to express within a thread block
 - Barriers operate on all the threads of a block
 - Relaxed via CUDA 9 cooperative groups
- Blocks must execute independently
 - Combining results computed by multiple blocks usually done by launching a second kernel on a new grid of blocks
- Recursive function calls
 - Limited by stack space
 - No static variable declarations inside the function
 - No variable number of arguments

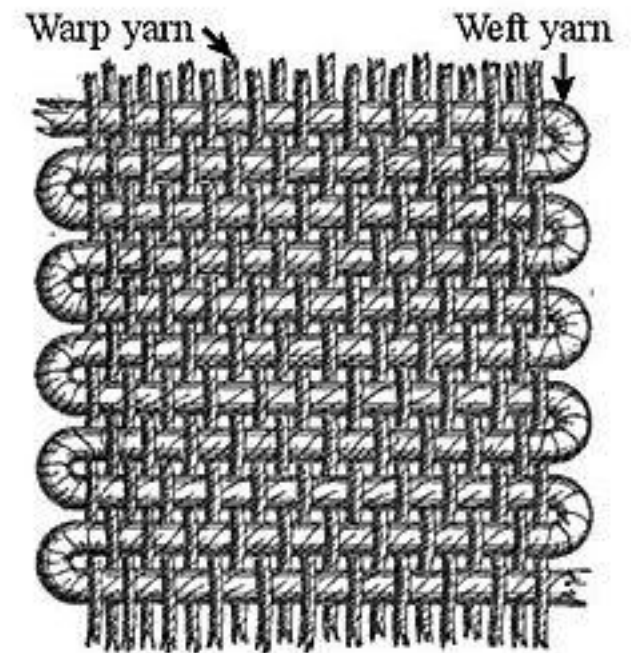
Restrictions (2)

- CUDA programs must copy data between host and device memory
 - Large compute-intensive problems better suited than smaller problems
 - CUDA 6 unified memory avoids explicit copying
 - On GraceHopper, CPU and GPU threads can now concurrently and transparently access both CPU- and GPU-resident memory

Single-instruction multiple-thread (SIMT) architecture

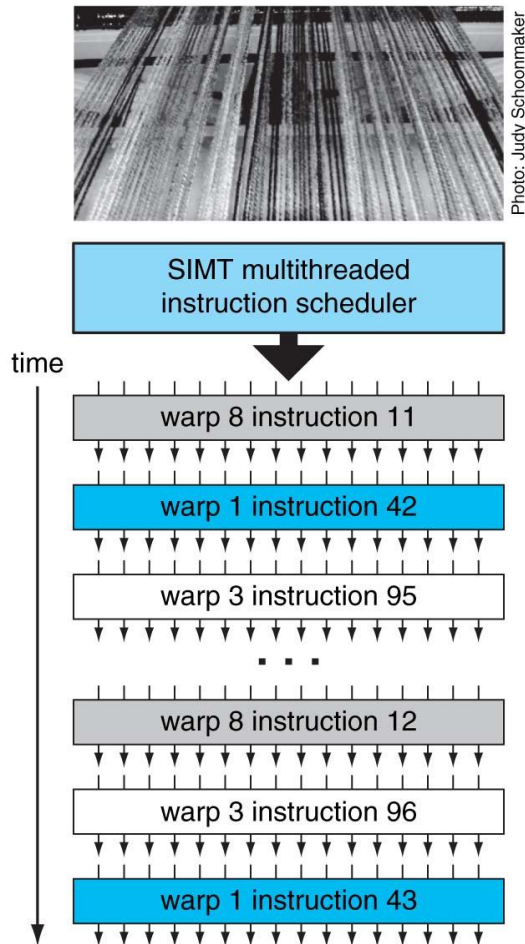
- Streaming multiprocessor creates, manages, schedules, and executes threads in groups of 32 parallel threads called **warps**
- At each instruction issue time
 - SIMT instruction unit selects warp that is ready to execute its next instruction
 - Broadcasts instruction to all active threads of that warp
 - Individual threads may be inactive to do independent branching

The term warp originates from weaving



Source: Wikipedia

Warp scheduling



- Typically, many more warps reside in a multiprocessor than can be executed truly simultaneously
 - Needed for **latency hiding**
 - Warps waiting for the completion of memory accesses, pipelined floating-point operations, or branch instructions are suspended
 - Warps ready for execution are resumed

Source: Patterson, Hennessy: Computer Organization & Design, 4th edition, Morgan Kaufmann



Zero-overhead thread scheduling

- While they are waiting, the status of threads is kept in register files
- Enables highly efficient scheduling
- Reason why GPUs do not invest as much chip area in caches as normal CPUs do
- As a result, GPUs can dedicate more chip area to floating-point execution resources

SIMT performance

- Full efficiency if all threads of a warp take the same execution path
- If threads of a warp diverge via a data-dependent conditional branch, execution serializes for each branch path taken
 - For equal-length paths, a divergent if-then-else block is 50% efficient
- May happen, for example,
 - At if-then-else statements if condition depends on thread identity
 - In for loops that iterate different number of times for different threads
- Not important for correctness but for performance
- Note: grouping threads into warps speeds up barrier synchronization

```
if (threadIdx.x == a)
    foo();
else
    bar();
```

SIMD vs. SIMT

SIMD

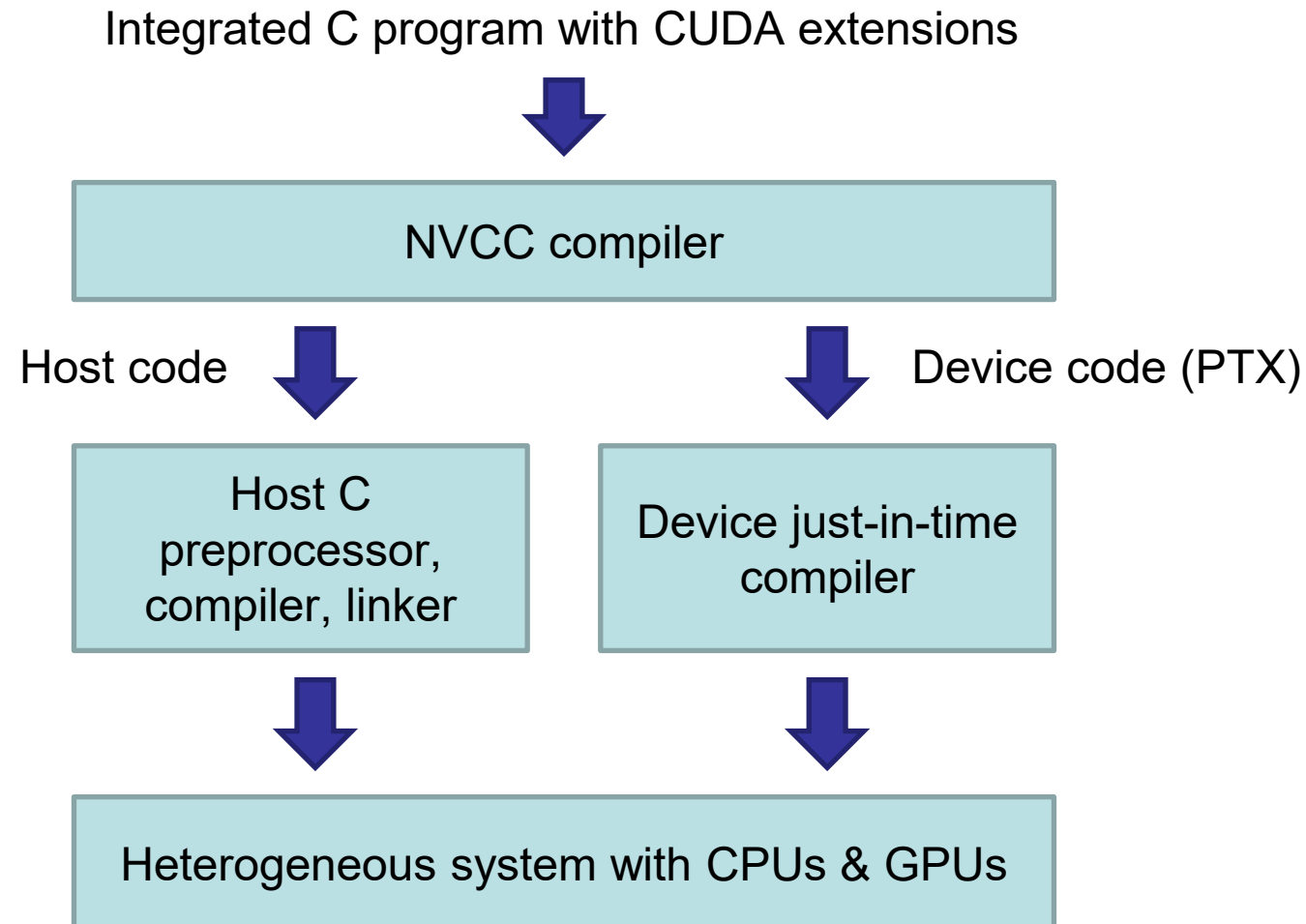
- Applies one instruction to multiple data lanes
- Requires software to express data parallelism explicitly in each vector instruction

SIMT

- Applies one instruction to multiple independent threads in parallel
- Finds data parallelism among independent threads
- Programs and instructions describe behavior of single independent thread

SIMT combines performance of SIMD with productivity of multithreading

Compilation workflow





Compilation workflow

Offline

- Compiler separates host from device code
- Compiles device code into assembly form (PTX) and/or binary form (cubin object)
- Modifies host code by replacing <<<>>> syntax (cuda kernel call) by runtime function calls
- Modified host code output either as C code left to be compiled by another tool or as object code

PTX = Parallel Thread Execution

Just-in-time

- PTX code loaded by an application at runtime is compiled further to binary code by device driver
- Increases application load time
- Allows application to benefit from latest compiler improvements
- Device driver caches copy of binary

Compatibility

- Binary compatibility
 - Binary code is architecture (i.e., compute-capability) specific
 - Compatibility guaranteed between minor revisions
- PTX compatibility
 - Some instructions only supported on devices of higher compute capability
- Application compatibility
 - Application must load binary or PTX code compatible with compute capability of device
 - To run code on future architectures for which no binary can be produced yet, an application must load PTX code to be compiled just-in-time

Compatibility (2)

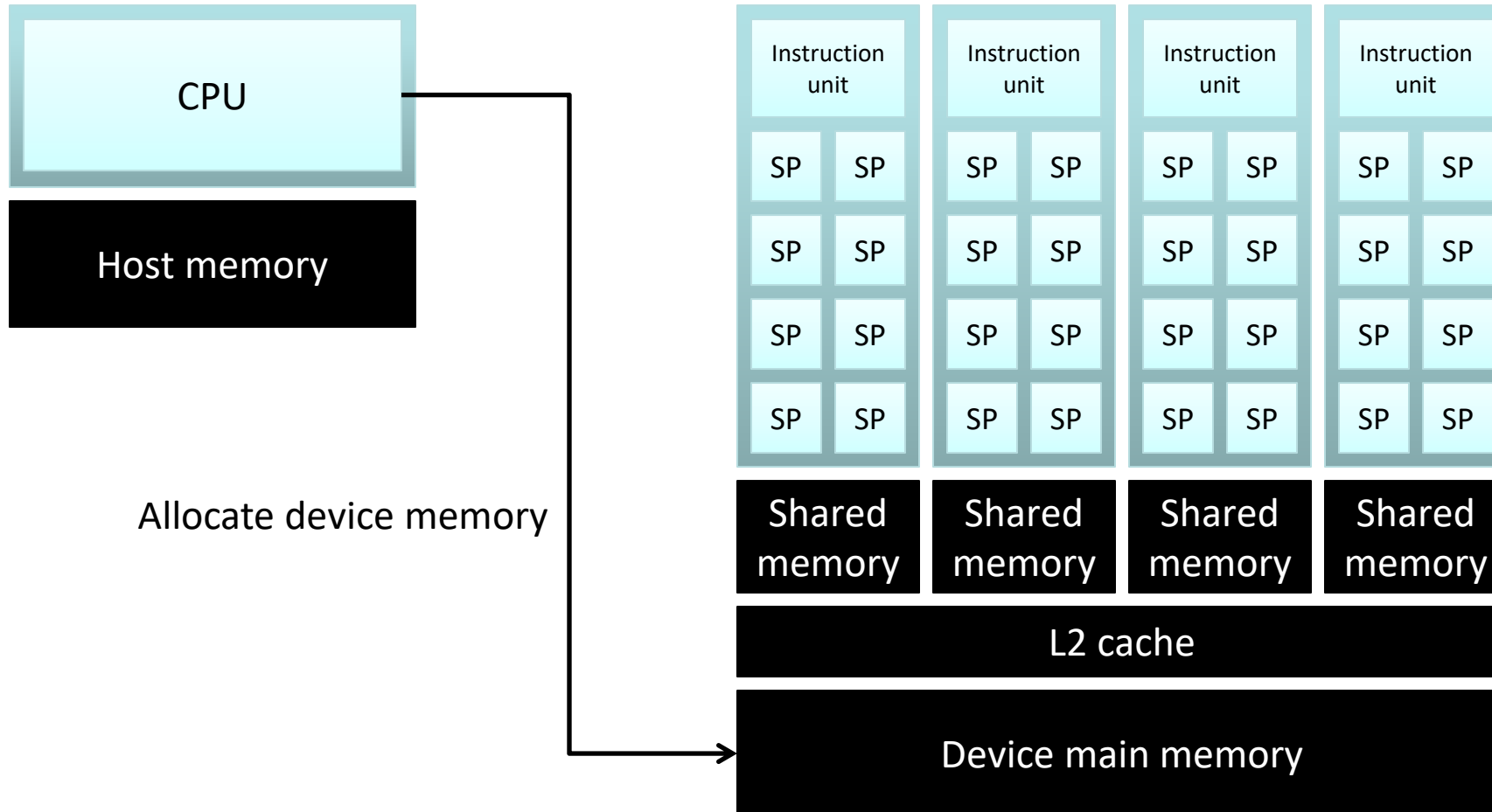
- C/C++ compatibility
 - Full C++ support for host code
 - Only subset of C++ supported for device code
- 64-bit compatibility
 - 64-bit device code only supported with 64-bit host code
 - 32-bit device code only supported with 32-bit host code



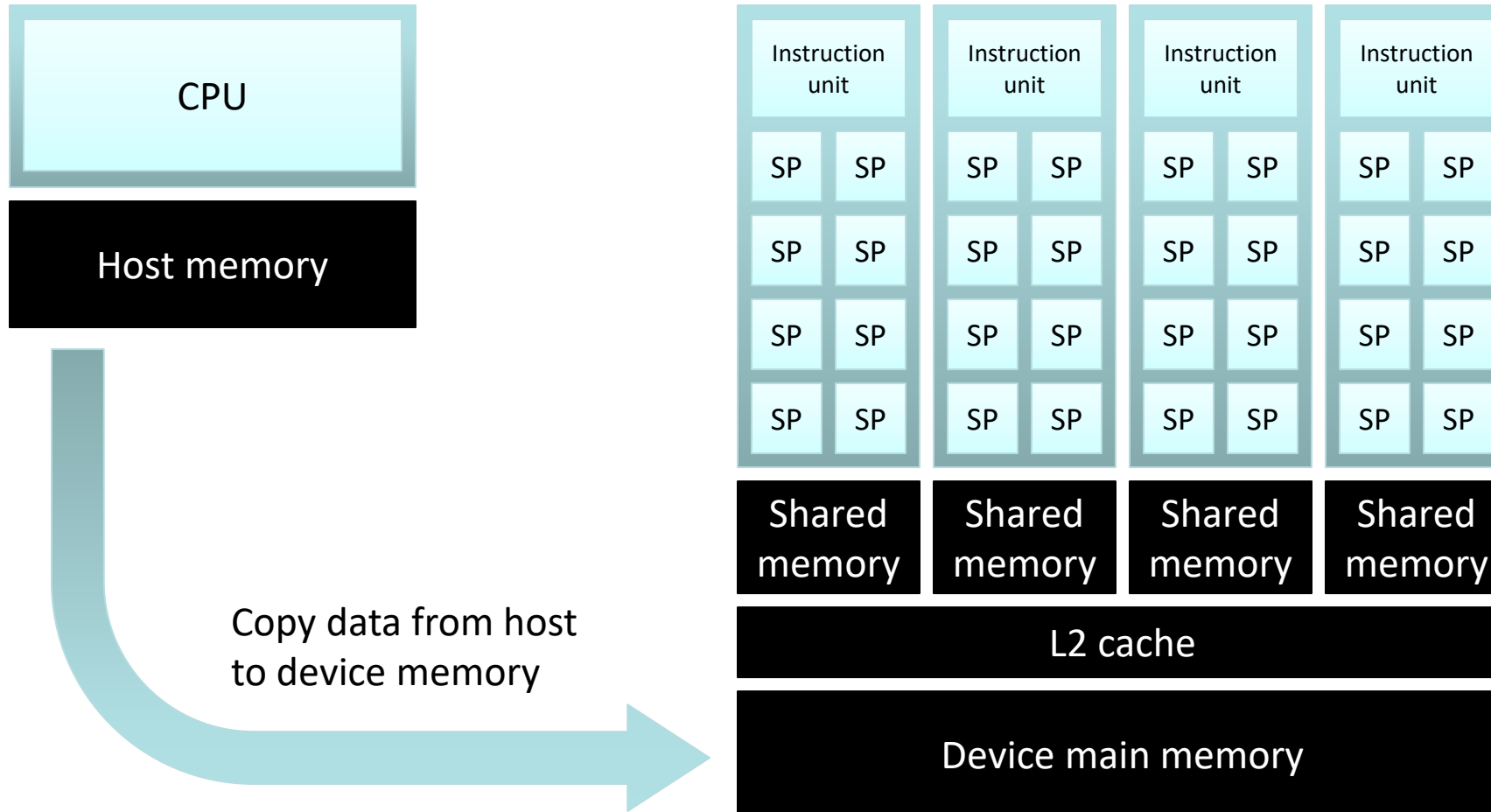
Instruction set architecture (ISA)

- To support multiple GPUs with different binary microinstruction formats, high-level language compilers generate intermediate assembler-level instructions
 - Direct3D vector instructions
 - PTX scalar instructions
- NVIDIA PTX (parallel thread execution) instruction set provides stable target ISA for compilers across multiple generations of GPUs

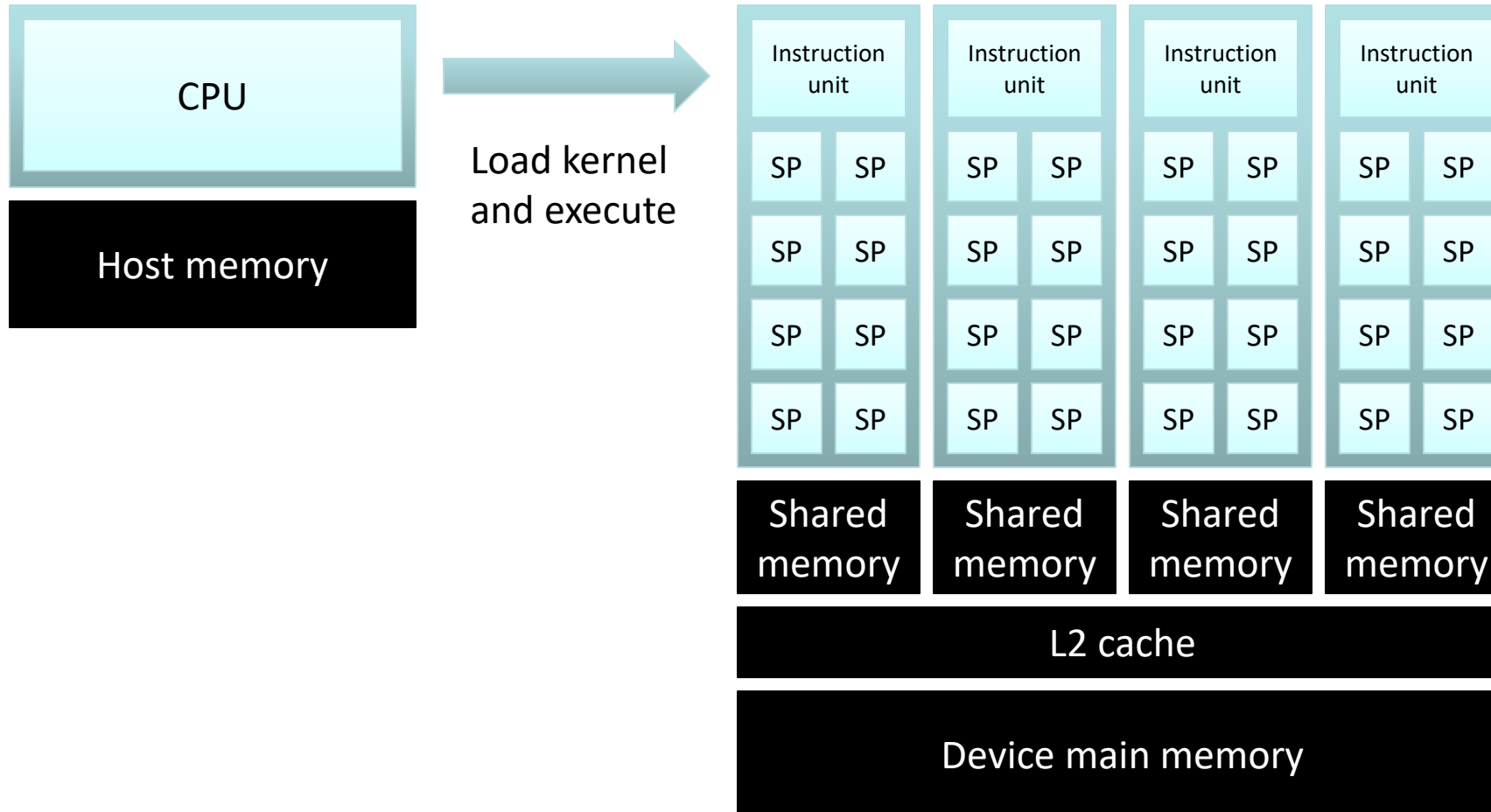
Executing a kernel



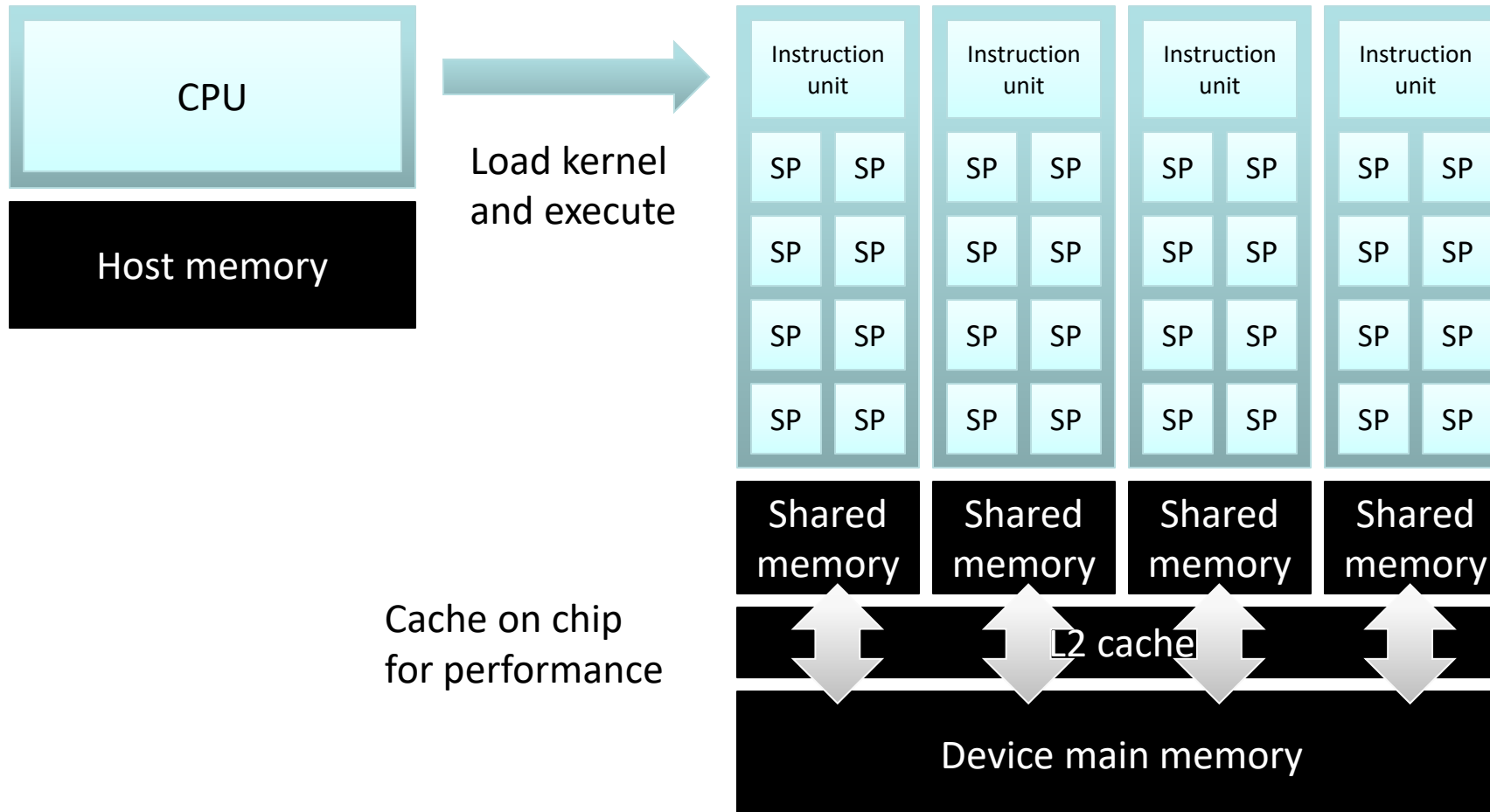
Executing a kernel



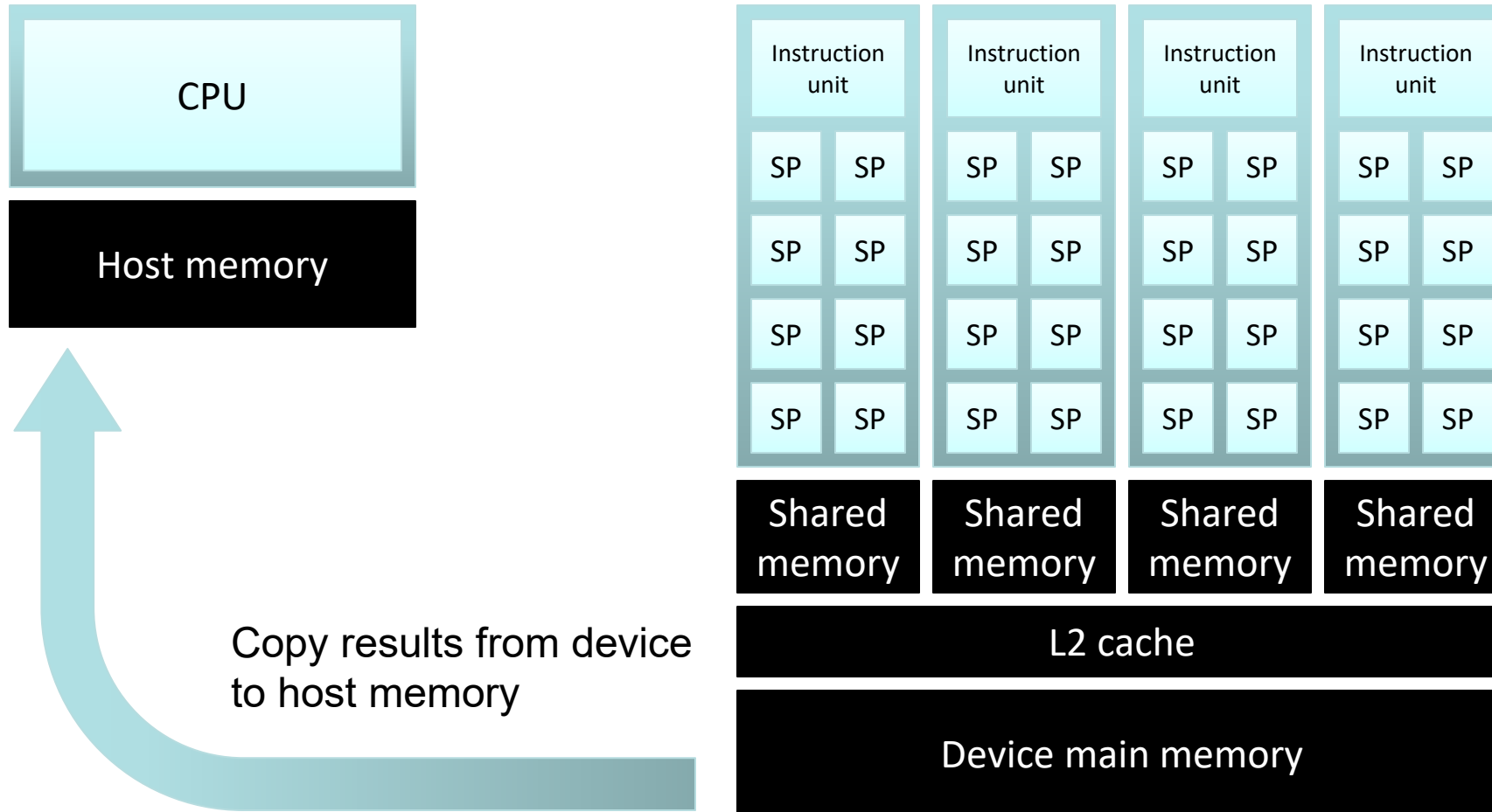
Executing a kernel



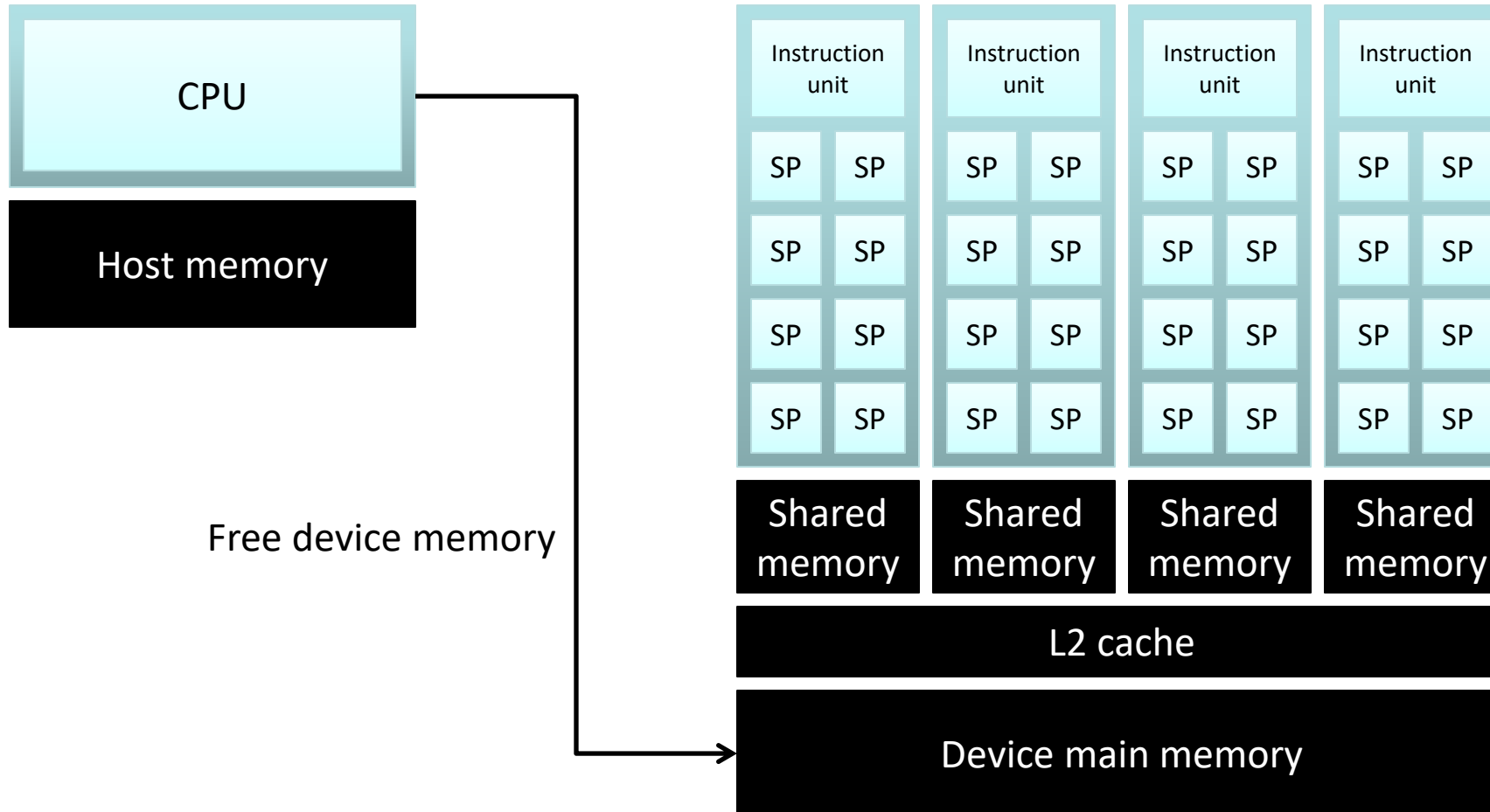
Executing a kernel



Executing a kernel



Executing a kernel



CUDA programming interface

Runtime with functions that execute on the host

- Allocate and free device memory
- Transfer data between host and device memory
- Manage systems with multiple devices

Extension of C(++) programming language

- Definition of kernel
- Launching kernel with specification of grid and block dimension

Initialization

- No explicit initialization function for runtime
 - Initializes implicitly the first time a runtime function is called
- During initialization, the runtime creates a CUDA context for each device in the system
- As part of this context creation, the device code is just-in-time compiled if necessary and loaded into device memory
- This all happens under the hood and the runtime does not expose the primary context to the application

Allocate memory on device

```
cudaError_t cudaMalloc(void **devPointer, size_t size)
```

- Allocates `size` bytes of linear memory on the device and returns in `*devPointer` a pointer to the allocated memory
- The allocated memory is suitably aligned for any type

Example

```
/* allocate a vector of 2048 floats on device */  
float *a_gpu;  
int n = 2048;  
cudaMalloc((void**) &a_gpu, n * sizeof(float));
```

Copy between host and device



```
cudaError_t cudaMemcpy(void *dst,  
                        const void *src,  
                        size_t count,  
                        enum cudaMemcpyKind kind)
```

- Copies count bytes from the memory area pointed to by **src** to the memory area pointed to by **dst**
- **kind** is one of
 - cudaMemcpyHostToHost
 - cudaMemcpyHostToDevice
 - cudaMemcpyDeviceToHost
 - cudaMemcpyDeviceToDevice
- The memory areas may not overlap

Example

Copy from host to device

```
/* copy vector a of floats of length n = 2048 from host to a_gpu on device */  
cudaMemcpy(a_gpu, a, n * sizeof(float), cudaMemcpyHostToDevice);
```

Copy from device to host

```
/* copy vector of floats a_gpu of length n = 2048 from device to a on host */  
cudaMemcpy(a, a_gpu, n * sizeof(float), cudaMemcpyDeviceToHost);
```

Reversed order

Changed flag



Free memory on device

```
cudaError_t cudaFree(void *devPointer)
```

Example

```
/* free the memory hold by a_gpu on device */  
cudaFree(a_gpu);
```

Error handling

```
const char* cudaGetErrorString(cudaError_t error)
```

Returns the message string from an error code



Calling a kernel

- Define dimensions (x,y,z) of thread block
- Define dimensions (x,y,z) of grid
- Call the kernel

```
// Setup the execution configuration
dim3 blockDim(blockSizeX, blockSizeY);
// Don't need to write z = 1
dim3 gridDim(gridSizeX, gridSizeY);

// Launch the kernel
Kernel<<<gridDim, blockDim>>>();
```

A first CUDA kernel: Hello CUDA!

Device code:

```
__global__ void helloCUDAKernel() {  
    printf(„Hello CUDA!\n“);  
}
```

Host code:

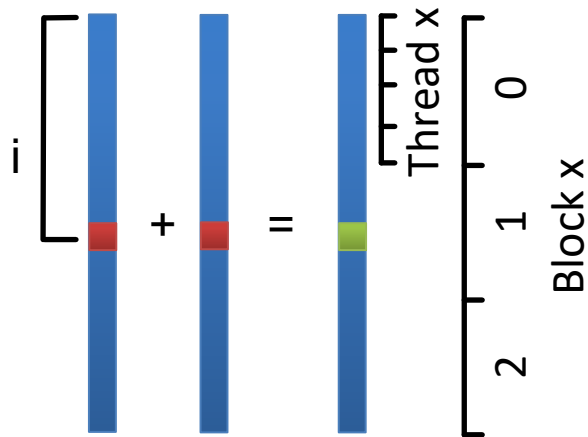
```
dim3 threads(BLOCK_SIZE);  
dim3 grid(GRID_SIZE);  
helloCUDAKernel <<< grid, threads >>>();
```

Creates the grid of blocks of threads
Does execute the Kernel on the GPU

Output:

- Each thread does print „Hello CUDA!“ (BLOCK_SIZE * GRID_SIZE lines)

Example: Vector addition



Serial CPU implementation

```
void add_vec_serial(int n, float *a , float *b , float *c) {  
    for (int i = 0; i < n; ++i) {  
        c[i] = a[i] + b[i];  
    }  
}
```

Parallel GPU implementation

```
__global__  
void add_vec_parallel(int n, float *a, float *b , float *c) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    if (i < n)  
        c[i] = a[i] + b[i];  
}
```

CUDA extensions to function declarations

	Executed on the	Only callable from the
<code>__device__ float DeviceFunc()</code>	device	device
<code>__global__ void KernelFunc()</code>	device	host
Default: <code>__host__ float HostFunc()</code>	host	host

Note that you can use both `__host__` and `__device__` in a function declaration



Built-in vector types

Vector types derived from the basic integer and floating-point types

- Structures with the 1st, 2nd, 3rd, and 4th components accessible through the fields `x`, `y`, `z`, and `w`, respectively
- Example: `int1`, `int2`, `int3`, `int4`
- Come with a constructor function of the form **`make_<type name>`**

```
int2 make_int2(int x, int y);
```

`dim3`

- Integer vector type based on `uint3` used to specify dimensions (`x`, `y`, `z`)
- Any component left unspecified is initialized to 1

Predefined variables

- Specify grid and block dimensions and block and thread indices
- Only valid within functions that are executed on the device

Variable	Semantics
dim3 gridDim	Dimensions of the grid
uint3 blockIdx	Block index within the grid
dim3 blockDim	Dimensions of the block
uint3 threadIdx	Thread index within the block
int warpSize	Warp size in threads

Thread organization

- Threads rely on unique coordinates to
 - Distinguish themselves from each other
 - Identify the appropriate portion of the data to process
- Threads are organized in two-level hierarchy
 - Grid = 3D array of blocks
 - Block = 3D array of threads
- Thread coordinates
 - **blockidx** = block index
 - **threadidx** = thread index within block

Thread organization (2)



Thread block 0

threadIdx

0	1	2	..	M-1
---	---	---	----	-----



```
int threadID = blockIdx.x *  
    blockDim.x + threadIdx.x;  
...  
float x = input[threadID];  
float y = func(x);  
output[threadID] = y;
```



Thread block N-1

threadIdx

0	1	2	..	M-1
---	---	---	----	-----



```
int threadID = blockIdx.x *  
    blockDim.x + threadIdx.x;  
...  
float x = input[threadID];  
float y = func(x);  
output[threadID] = y;
```



...

Global 3D coordinates

$$x = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

$$y = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$$

$$z = \text{blockIdx.z} * \text{blockDim.z} + \text{threadIdx.z}$$

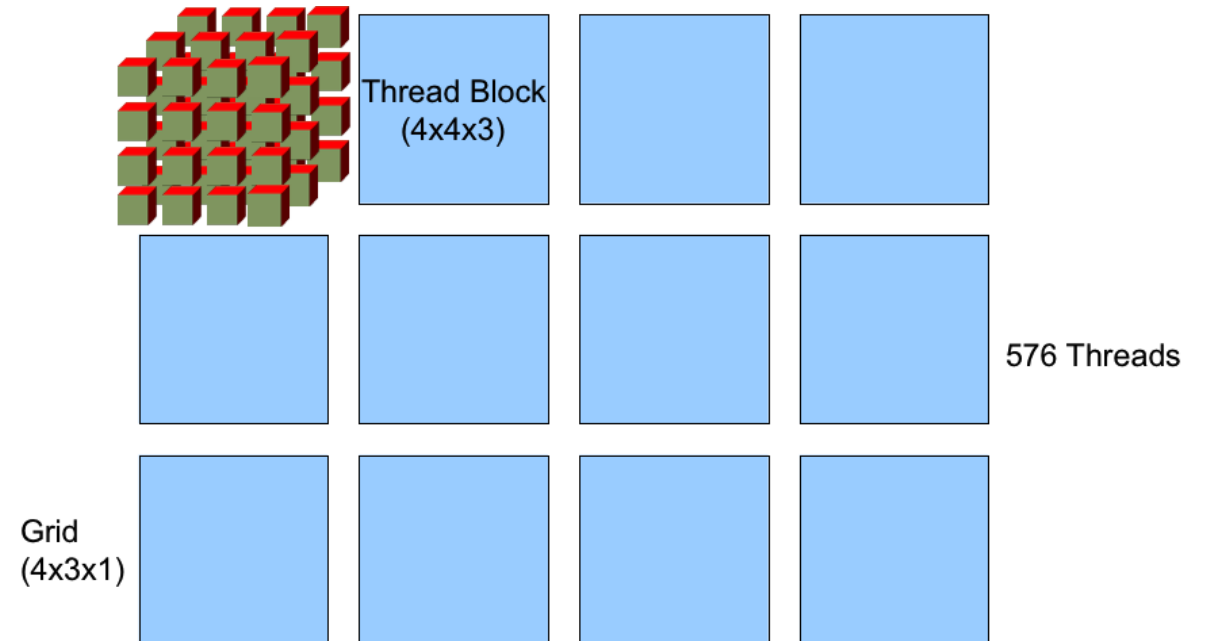
Launching multidimensional grid



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

```
dim3 gridDim(4, 3);  
dim3 blockDim(4, 4, 3);  
kernel<<<gridDim, blockDim>>>(...);  
// or  
kernel<<<{4, 3}, {4, 4, 3}>>>(...);
```

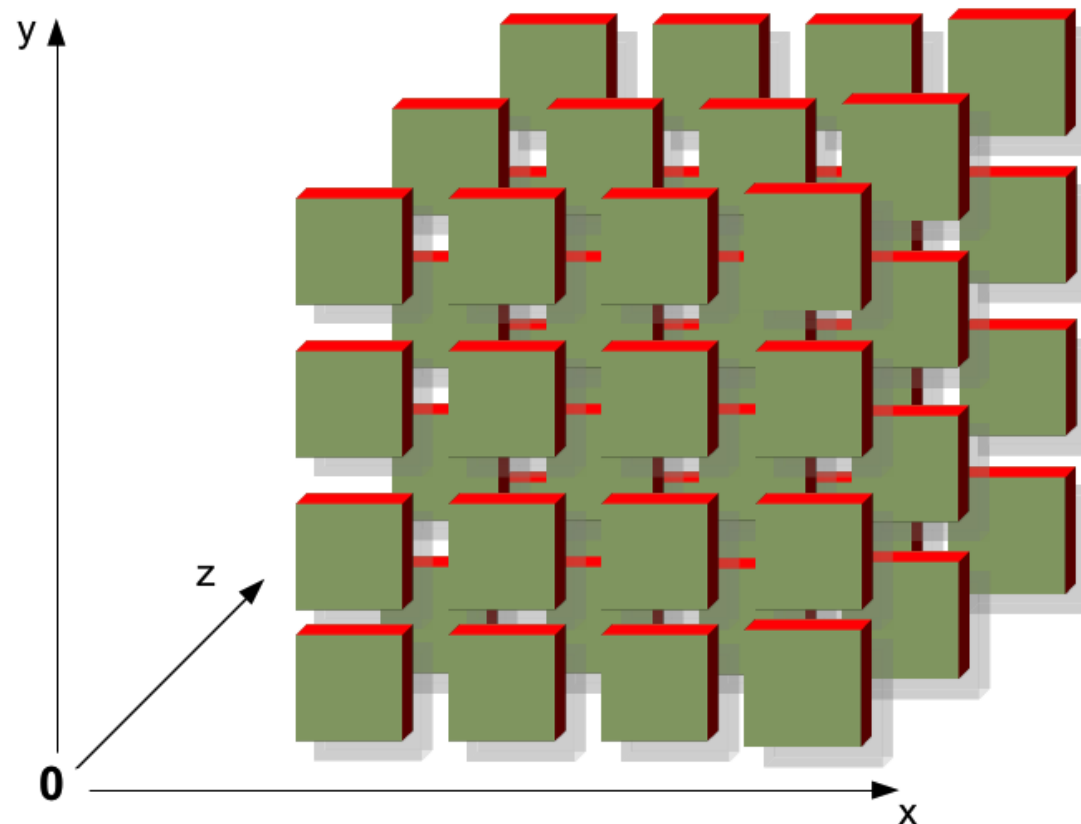


Thread block (4,4,3)



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming





Synchronizing CPU and GPU

- All kernel launches are asynchronous
 - Control returns to CPU immediately
 - Kernel starts executing once all previous CUDA calls have completed
- Memcopies are synchronous
 - Control returns to CPU once the copy is complete
 - Copy starts once all previous CUDA calls have completed
- `cudaDeviceSynchronize()` blocks until all previous CUDA calls complete
- Asynchronous CUDA calls provide
 - Non-blocking memcopies
 - Ability to overlap memcopies and kernel execution

Streams

- Stream – a sequence of operations that execute in issue-order on the GPU
 - CUDA operations in different streams may run concurrently
 - CUDA operations from different streams may be interleaved
- Default stream – stream used when no stream is specified
- Completely synchronous w.r.t. host and device
- Exceptions
 - Kernel launches in the default stream
 - `cudaMemcpyAsync`
 - `cudaMemsetAsync`
 - `cudaMemcpy` within the same device
 - H2D `cudaMemcpy` of 64kB or less

Here, we always use the default stream



Barrier synchronization

```
void __syncthreads()
```

- Waits until all threads in the thread block have reached this point and all global and shared memory accesses made by these threads prior to **__syncthreads()** are visible to all threads in the block
- Helps to avoid potential read-after-write, write-after-read, or write-after-write hazards

Example for barrier synchronization



```
__global__ void stencil_1d(int *in, int *out) {
    __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;
    int lindex = threadIdx.x + RADIUS;

    // Read input elements into shared memory
    temp[lindex] = in[gindex];
    if (threadIdx.x < RADIUS) {
        temp[lindex - RADIUS] = in[gindex - RADIUS];
        temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
    }

    // Synchronize (ensure all the data is available)
    __syncthreads();

    // Apply the stencil
    int result = 0;
    for (int offset = -RADIUS; offset <= RADIUS; offset++)
        result += temp[lindex + offset];

    // Store the result
    out[gindex] = result;
}
```

- Without barrier, reading of temp array may not be finished when stencil application begins
- Threads in a block execute logically in parallel, but not all threads can execute physically at the same time
- Without sync, data race between threads in different warps



Summary CUDA threads

- Kernel execution configuration defines dimensions of a grid and its blocks
- Unique coordinates in blockIdx and threadIdx variables allow threads to identify themselves and their data
- Blocks are assigned to SMs
- Transparent scalability comes from the limitation that threads in different blocks cannot synchronize
- Blocks are partitioned into warps
 - Usually, only a subset of the warps are executed at a time
 - Enables latency hiding

Example – square matrix multiplication



```
void MatrixMultiplication(float *M, float *N, float *P, int Width)
{
    int size = Width * Width * sizeof(float);
    float *Md, *Nd, *Pd;

    // Transfer M and N to device memory
    cudaMalloc((void**) &Md, size);
    cudaMemcpy(Md, M, size, cudaMemcpyHostToDevice);
    cudaMalloc((void**) &Nd, size);
    cudaMemcpy(Nd, N, size, cudaMemcpyHostToDevice);
    // Allocate P on device
    cudaMalloc((void**) &Pd, size);

    // Kernel invocation code – to be shown later
    [...]

    // Transfer P from device to host
    cudaMemcpy(P, Pd, size, cudaMemcpyDeviceToHost);
    // Free device matrices
    cudaFree(Md); cudaFree(Nd); cudaFree(Pd);
}
```

Kernel

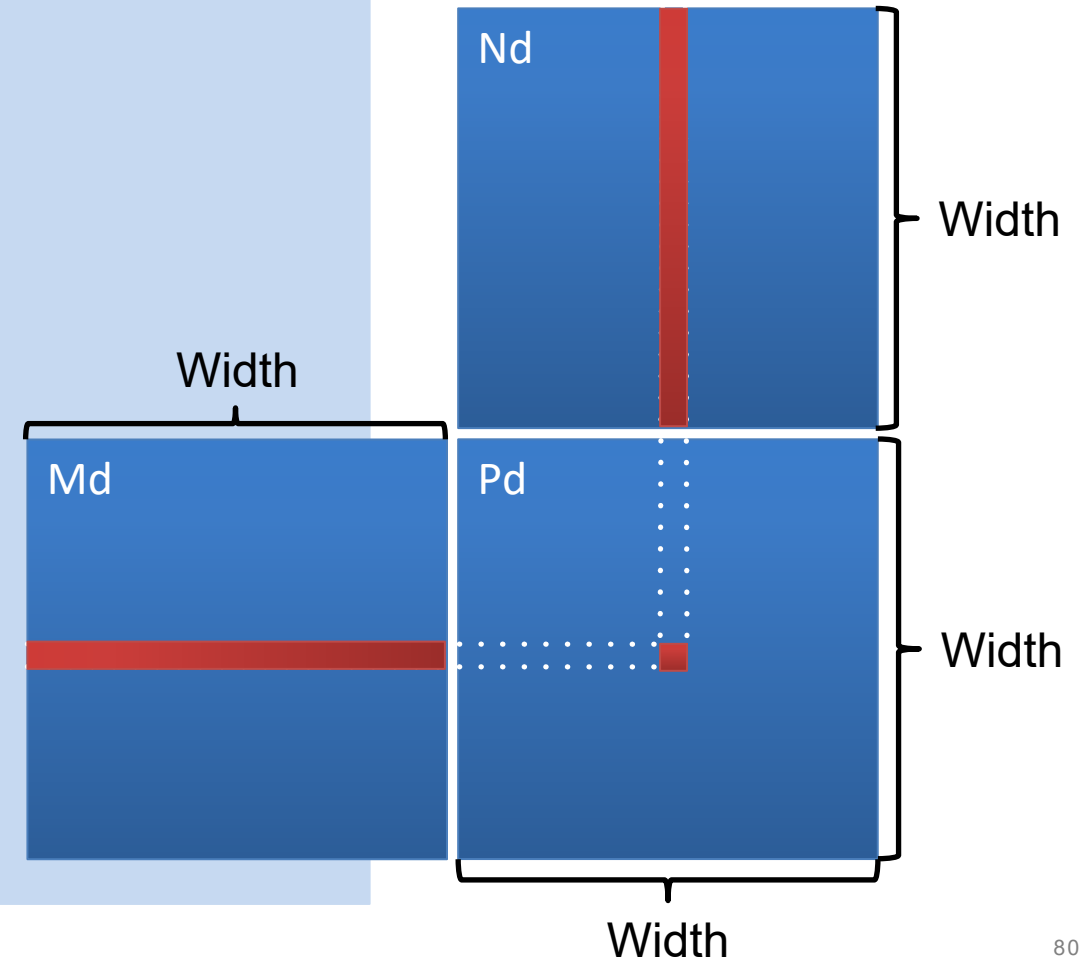


```
__global__
void MatrixMulKernel(float* Md, float* Nd, float* Pd, int Width)
{
    // 2D thread ID – assuming a single block of threads
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // Pvalue stores element computed by the thread
    float Pvalue = 0;

    for (int k = 0; k < Width; ++k){
        float Mdelement = Md[ty * Width + k];
        float Ndelement = Nd[k * Width + tx];
        Pvalue += Mdelement * Ndelement;
    }

    // Write the matrix to device memory
    // Each thread writes one element
    Pd[ty * Width + tx] = Pvalue;
}
```





Calling the kernel

- Define dimensions of thread block
- Define dimensions of grid
- Call the kernel

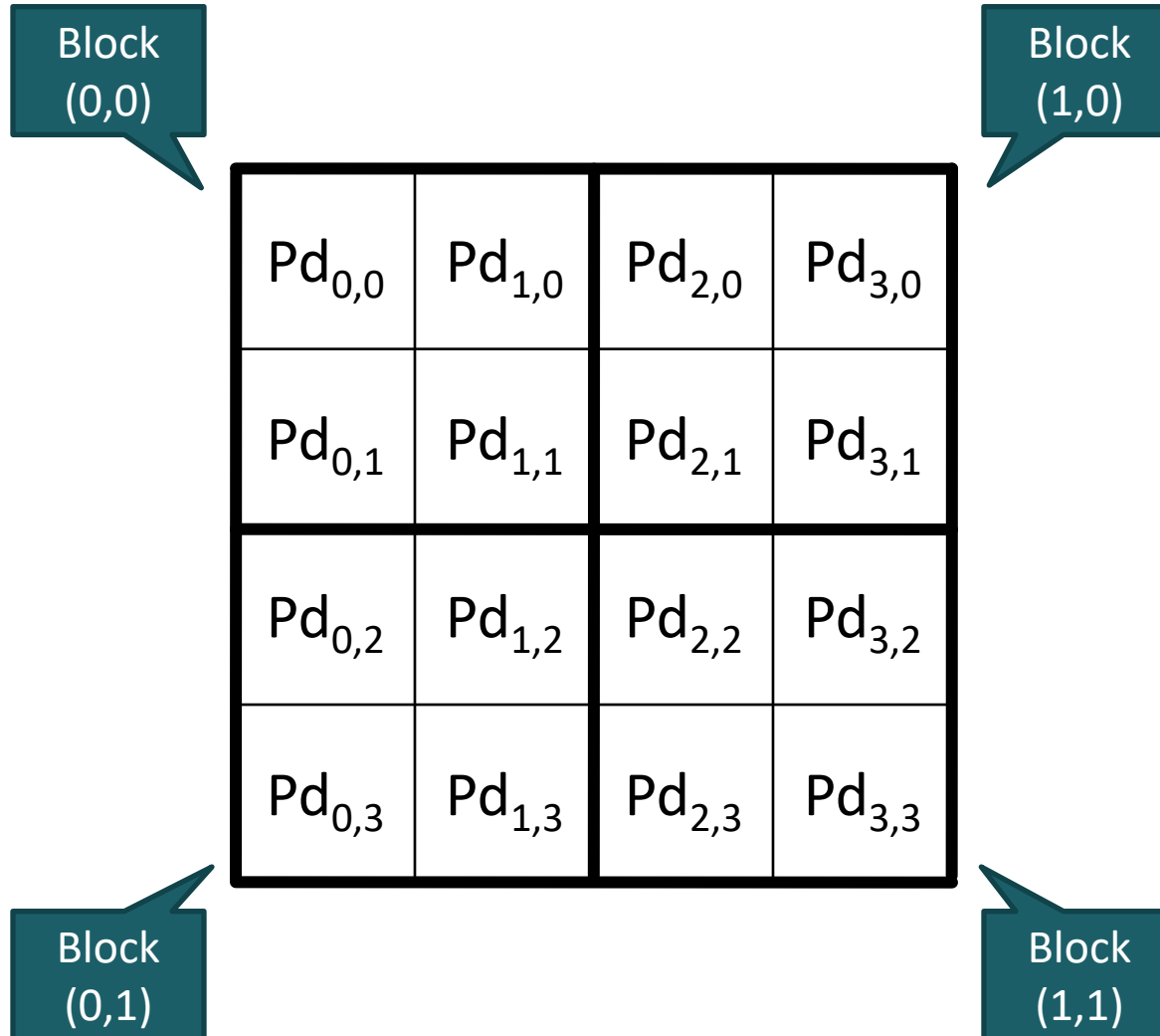
```
// Setup the execution configuration
dim3 blockDim(Width, Width); // Don't need to write z = 1
dim3 gridDim(1, 1);

// Launch the kernel
MatrixMulKernel<<<gridDim, blockDim>>>(Md, Nd, Pd, Width);
```

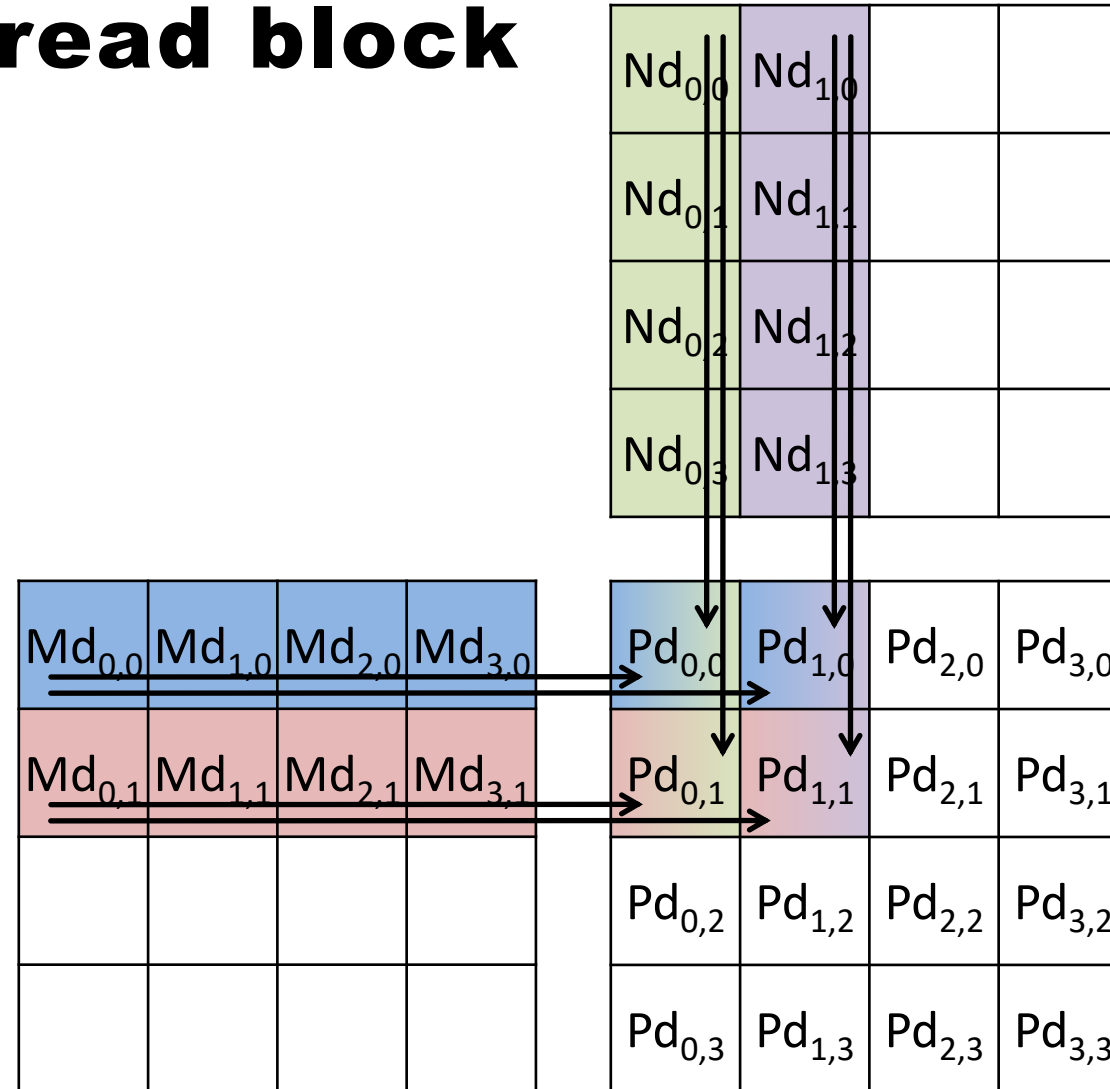
Discussion

- Previous version assumed single block of threads
 - Number of matrix elements limited to maximum number of threads per block
 - Limited scalability
- Idea
 - Break result matrix into tiles
 - All elements of a tile are computed by a block of threads

Simplified example of tiling Pd



Matrix multiplication actions of one thread block



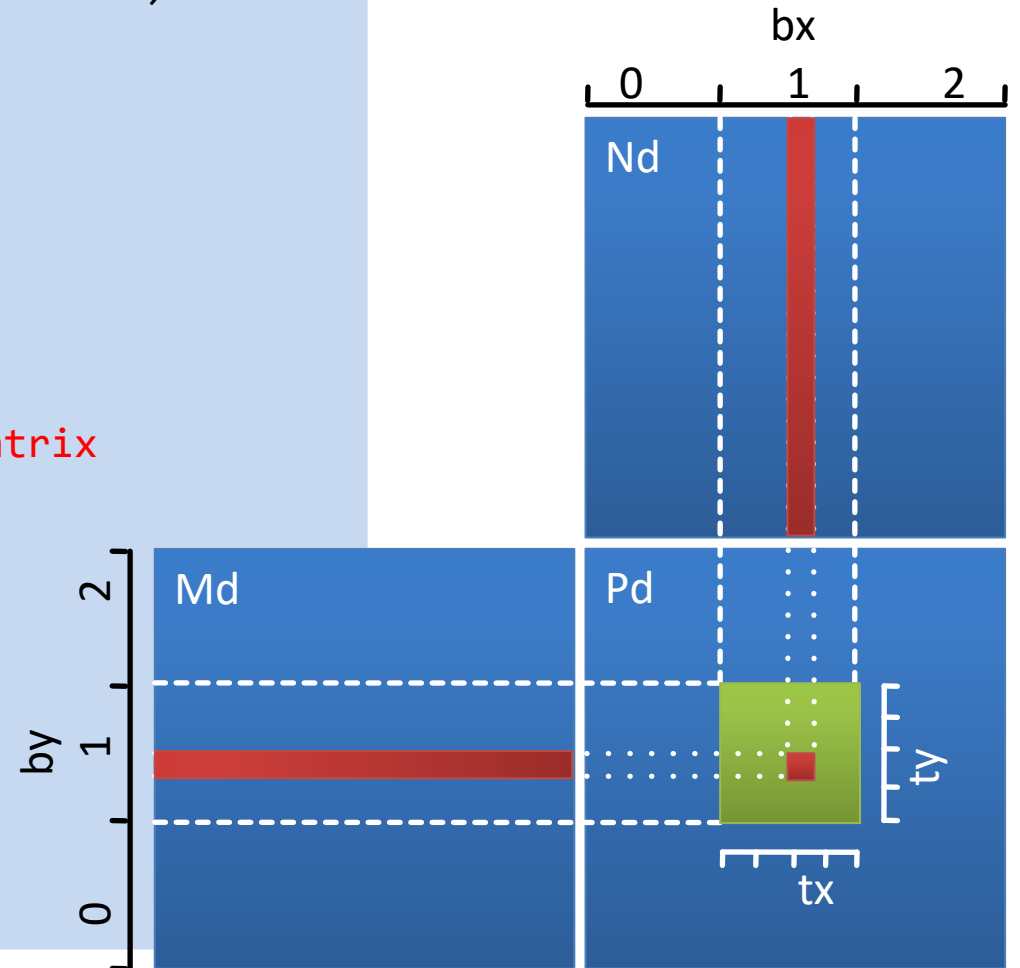
Result matrix divided into tiles



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

```
__global__  
void MatrixMulKernel(float* Md, float* Nd, float* Pd, int Width)  
{  
    // Calculate the row index of the Pd element and Md  
    int Row = blockIdx.y * TILE_WIDTH + threadIdx.y;  
    // Calculate the column index of the Pd element and Nd  
    int Col = blockIdx.x * TILE_WIDTH + threadIdx.x;  
  
    float Pvalue = 0;  
  
    // each thread computes one element of the block sub-matrix  
    for (int k = 0; k < Width; ++k)  
        float Mdelement = Md[Row * Width + k];  
        float Ndelement = Nd[k * Width + Col];  
        Pvalue += Mdelement * Ndelement;  
  
    // Write the matrix to device memory  
    // Each thread writes only one element  
    Pd[Row * Width + Col] = Pvalue;  
}
```



Host code

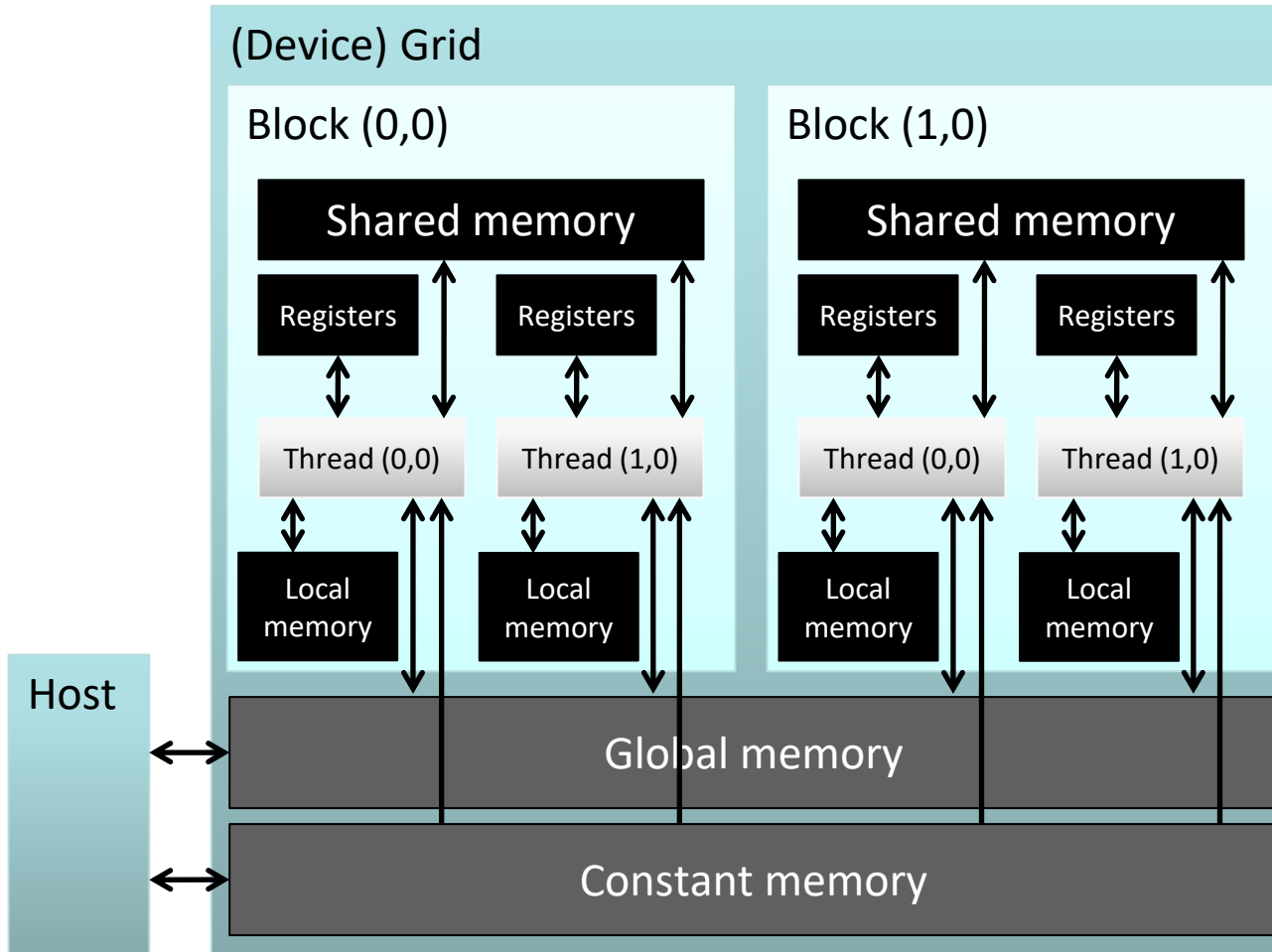


```
// Setup the execution configuration
dim3 blockDim(TILE_WIDTH, TILE_WIDTH);
dim3 gridDim(Width/TILE_WIDTH, Width/TILE_WIDTH);

// Launch the kernel
MatrixMulKernel<<<gridDim, blockDim>>>(Md, Nd, Pd, Width);
```

We assume that Width
is multiple of
TILE_WIDTH

Reiterate CUDA device memories



Each thread can:

- R/W per-thread **registers**
- R/W per-thread **local memory**
- R/W per-block **shared memory**
- R/W per-grid **global memory**
- Read only per-grid **constant memory**

The host can

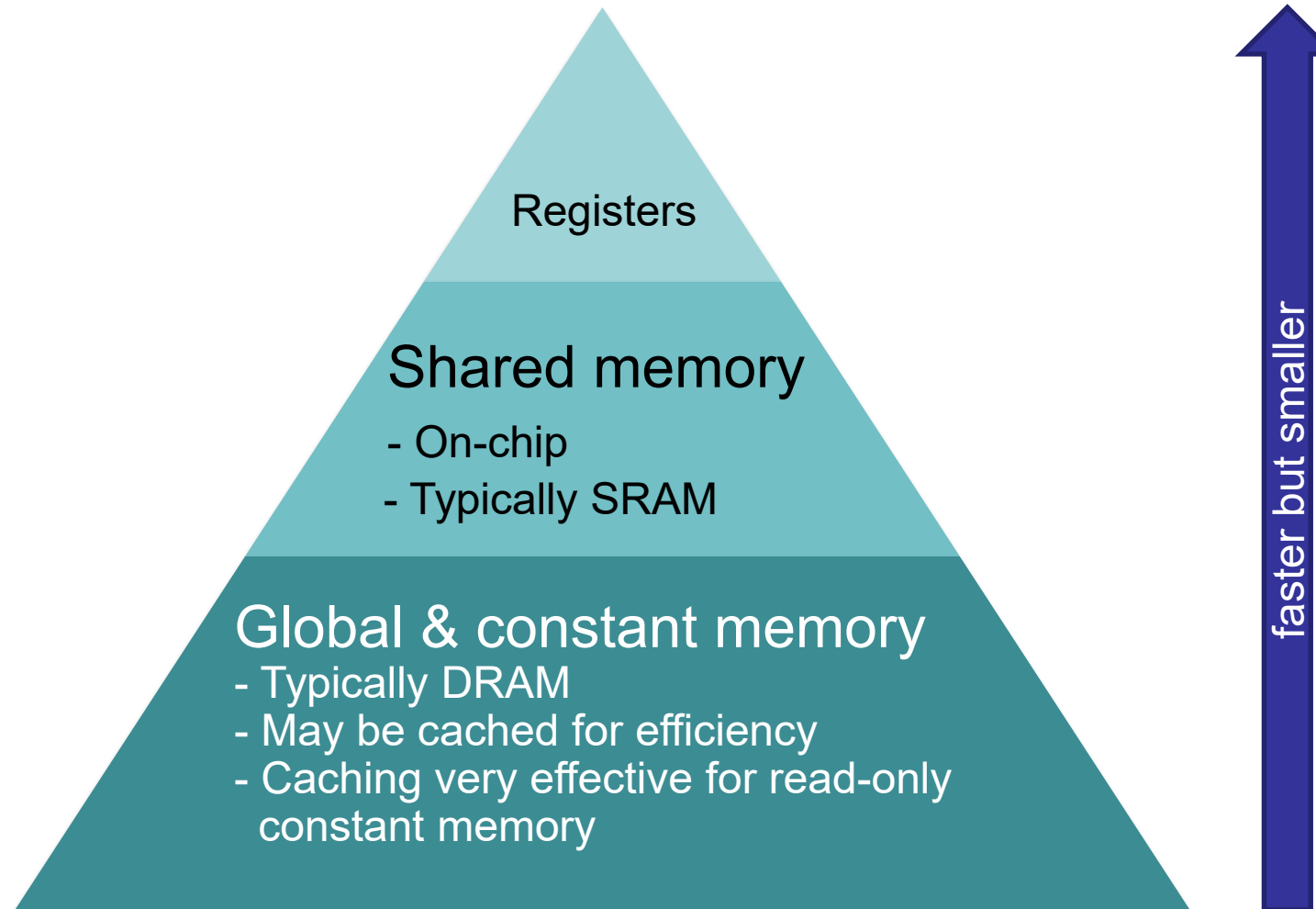
- R/W **global memory**
- R/W **constant memory**

Memory speed



TECHNISCHE
UNIVERSITÄT
DARMSTADT

 Parallel
 Programming

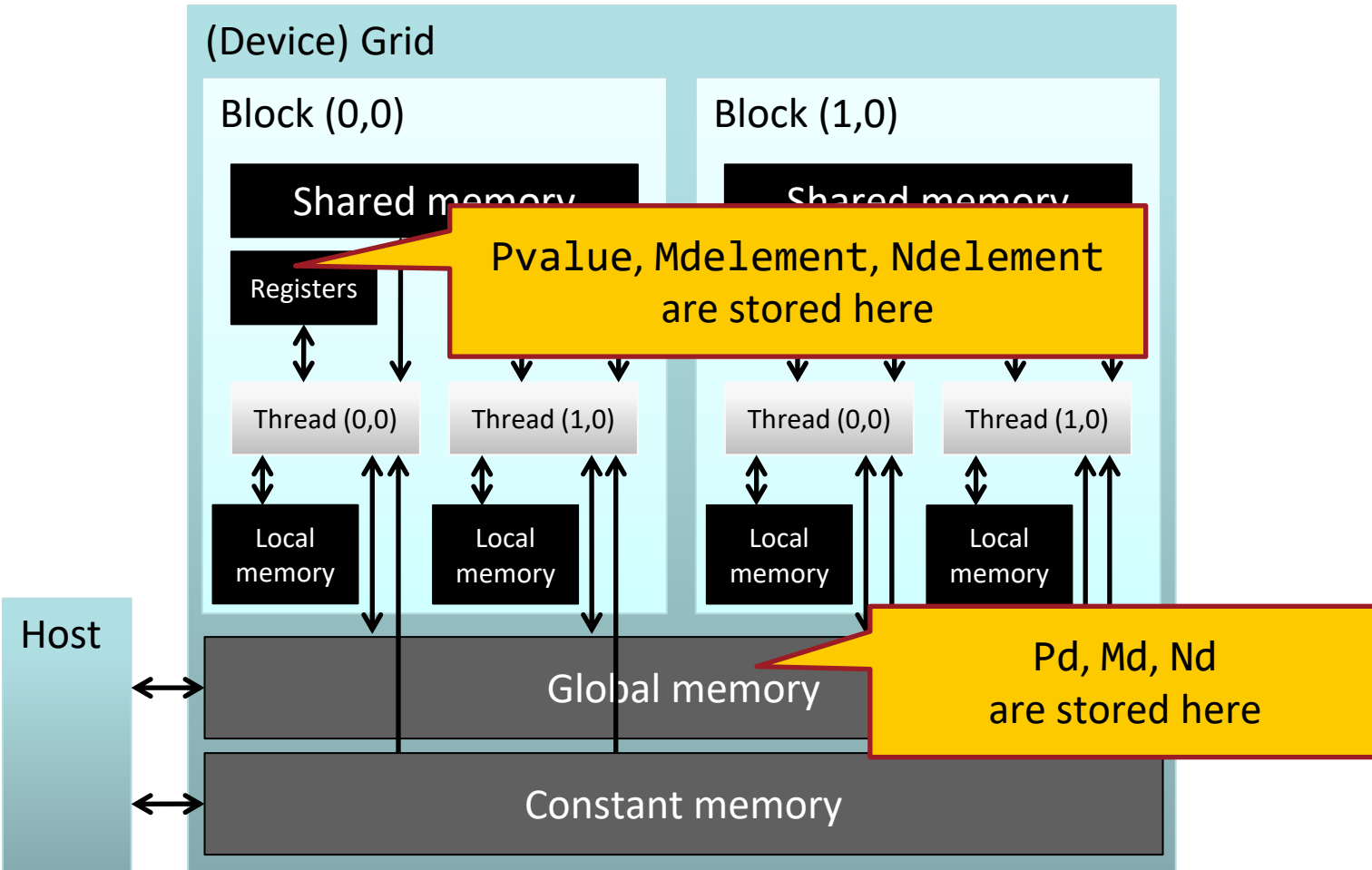


CUDA variable type qualifiers

Variable declaration	Memory	Scope	Lifetime
Automatic scalar variables	Register	Thread	Kernel
Automatic array variables	Local	Thread	Kernel
<code>__shared__ int SharedVar;</code>	Shared	Block	Kernel
<code>__device__ int GlobalVar;</code>	Global	Grid	Application
<code>__constant__ int ConstantVar;</code>	Constant	Grid	Application

Declaration only outside function bodies

Reiterate CUDA device memories



Each thread can:

- R/W per-thread **registers**
- R/W per-thread **local memory**
- R/W per-block **shared memory**
- R/W per-grid **global memory**
- Read only per-grid **constant memory**

The host can

- R/W **global memory**
- R/W **constant memory**

Memory performance of matrix multiplication



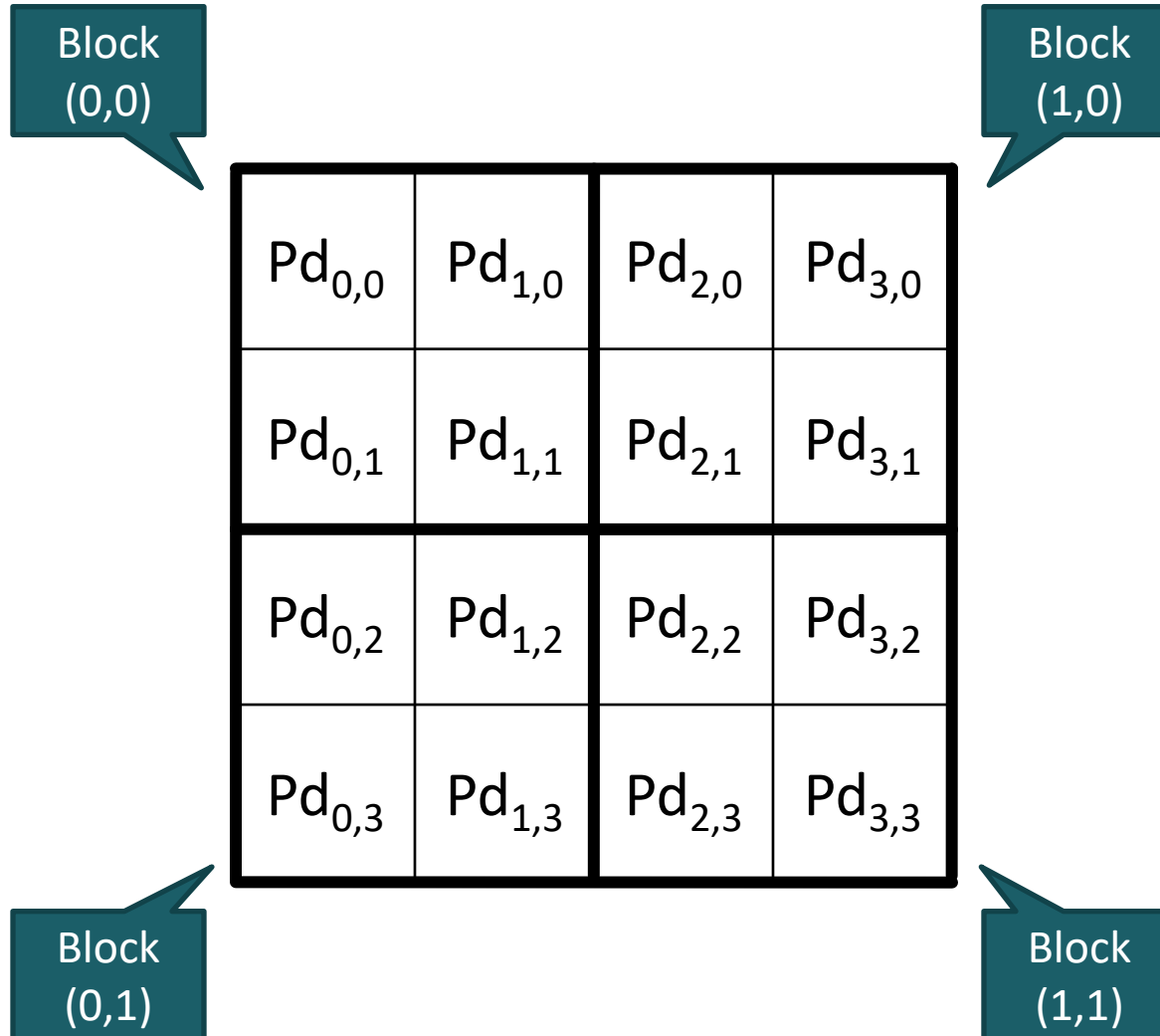
```
float Pvalue = 0;
for (int k = 0; k < Width; ++k) {
    float Mdelement = Md[Row * Width + k];
    float Ndelement = Nd[k * Width + Col];
    Pvalue += Mdelement * Ndelement;
}
```

FLOPs/byte = 0.25

Every iteration, we perform two loads and two float operations
=> Ratio 1.0

Called compute to global memory access (CGMA) ratio

Simplified example of tiling Pd



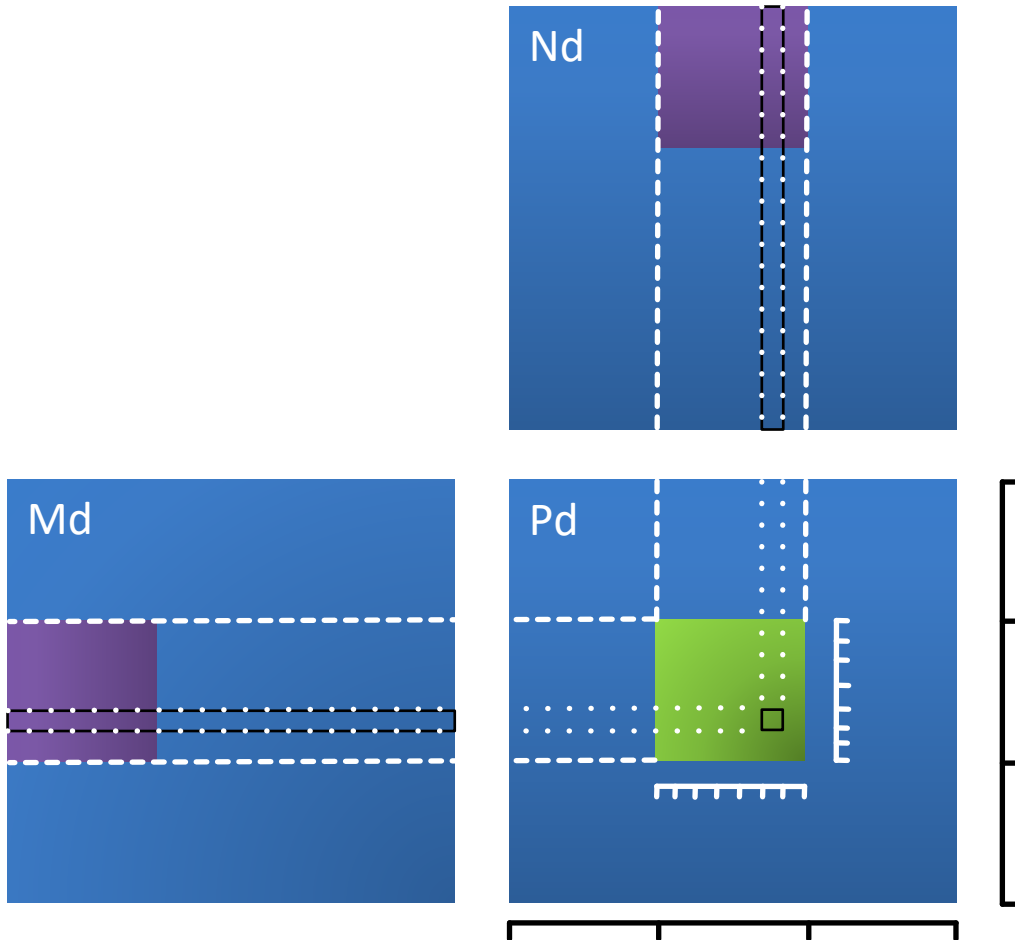
Global memory accesses performed by threads in block (0,0)



Access order ↓

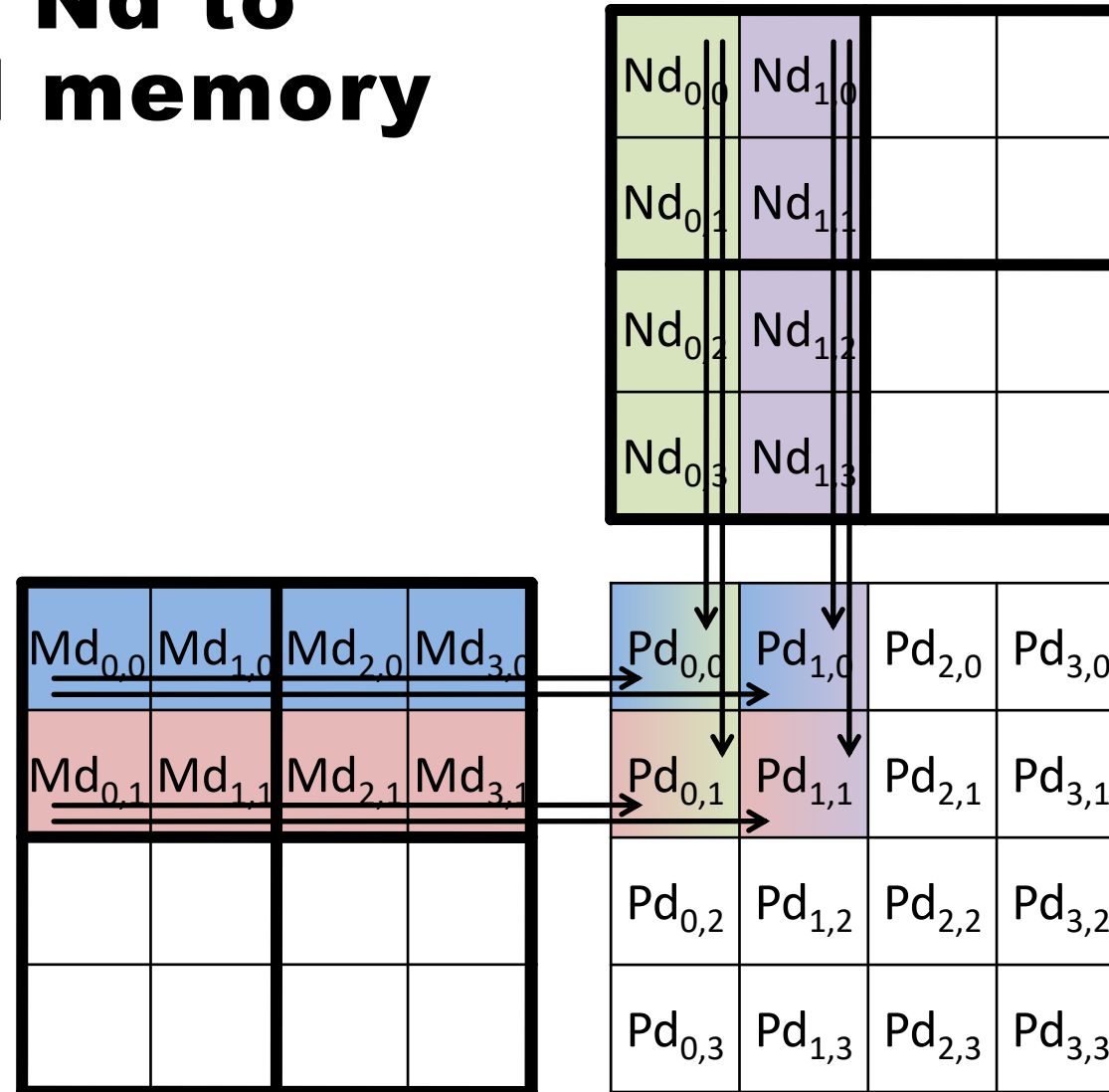
$Pd_{0,0}$ Thread(0,0)	$Pd_{1,0}$ Thread(1,0)	$Pd_{0,1}$ Thread(0,1)	$Pd_{1,1}$ Thread(1,1)
$Md_{0,0} * Nd_{0,0}$	$Md_{0,0} * Nd_{1,0}$	$Md_{0,1} * Nd_{0,0}$	$Md_{0,1} * Nd_{1,0}$
$Md_{1,0} * Nd_{0,1}$	$Md_{1,0} * Nd_{1,1}$	$Md_{1,1} * Nd_{0,1}$	$Md_{1,1} * Nd_{1,1}$
$Md_{2,0} * Nd_{0,2}$	$Md_{2,0} * Nd_{1,2}$	$Md_{2,1} * Nd_{0,2}$	$Md_{2,1} * Nd_{1,2}$
$Md_{3,0} * Nd_{0,3}$	$Md_{3,0} * Nd_{1,3}$	$Md_{3,1} * Nd_{0,3}$	$Md_{3,1} * Nd_{1,3}$

Optimization



- Currently, each value loaded from global memory is used only once
- How to increase flops/byte ratio in global memory?
- Store elements in shared memory. **But which elements?**
- Partition (input) data into tiles that fit into shared memory
- Important: computations on tiles must be done independently of each other

Tiling Md and Nd to utilize shared memory



Execution phases of tiled matrix multiplication



	Phase 1			Phase2		
$T_{0,0}$	$\mathbf{Md}_{0,0}$ ↓ Mds _{0,0}	$\mathbf{Nd}_{0,0}$ ↓ Nds _{0,0}	$\text{Pvalue}_{0,0} +=$ $\text{Mds}_{0,0} * \text{Nds}_{0,0} +$ $\text{Mds}_{1,0} * \text{Nds}_{0,1}$	$\mathbf{Md}_{2,0}$ ↓ Mds _{0,0}	$\mathbf{Nd}_{0,2}$ ↓ Nds _{0,0}	$\text{Pvalue}_{0,0} +=$ $\text{Mds}_{0,0} * \text{Nds}_{0,0} +$ $\text{Mds}_{1,0} * \text{Nds}_{0,1}$
$T_{1,0}$	$\mathbf{Md}_{1,0}$ ↓ Mds _{1,0}	$\mathbf{Nd}_{1,0}$ ↓ Nds _{1,0}	$\text{Pvalue}_{1,0} +=$ $\text{Mds}_{0,0} * \text{Nds}_{1,0} +$ $\text{Mds}_{1,0} * \text{Nds}_{1,1}$	$\mathbf{Md}_{3,0}$ ↓ Mds _{1,0}	$\mathbf{Nd}_{1,2}$ ↓ Nds _{1,0}	$\text{Pvalue}_{1,0} +=$ $\text{Mds}_{0,0} * \text{Nds}_{1,0} +$ $\text{Mds}_{1,0} * \text{Nds}_{1,1}$
$T_{0,1}$	$\mathbf{Md}_{0,1}$ ↓ Mds _{0,1}	$\mathbf{Nd}_{0,1}$ ↓ Nds _{0,1}	$\text{Pvalue}_{0,1} +=$ $\text{Mds}_{0,1} * \text{Nds}_{0,0} +$ $\text{Mds}_{1,1} * \text{Nds}_{0,1}$	$\mathbf{Md}_{2,1}$ ↓ Mds _{0,1}	$\mathbf{Nd}_{0,3}$ ↓ Nds _{0,1}	$\text{Pvalue}_{0,1} +=$ $\text{Mds}_{0,1} * \text{Nds}_{0,0} +$ $\text{Mds}_{1,1} * \text{Nds}_{0,1}$
$T_{1,1}$	$\mathbf{Md}_{1,1}$ ↓ Mds _{1,1}	$\mathbf{Nd}_{1,1}$ ↓ Nds _{1,1}	$\text{Pvalue}_{1,1} +=$ $\text{Mds}_{0,1} * \text{Nds}_{1,0} +$ $\text{Mds}_{1,1} * \text{Nds}_{1,1}$	$\mathbf{Md}_{3,1}$ ↓ Mds _{1,1}	$\mathbf{Nd}_{1,3}$ ↓ Nds _{1,1}	$\text{Pvalue}_{1,1} +=$ $\text{Mds}_{0,1} * \text{Nds}_{1,0} +$ $\text{Mds}_{1,1} * \text{Nds}_{1,1}$

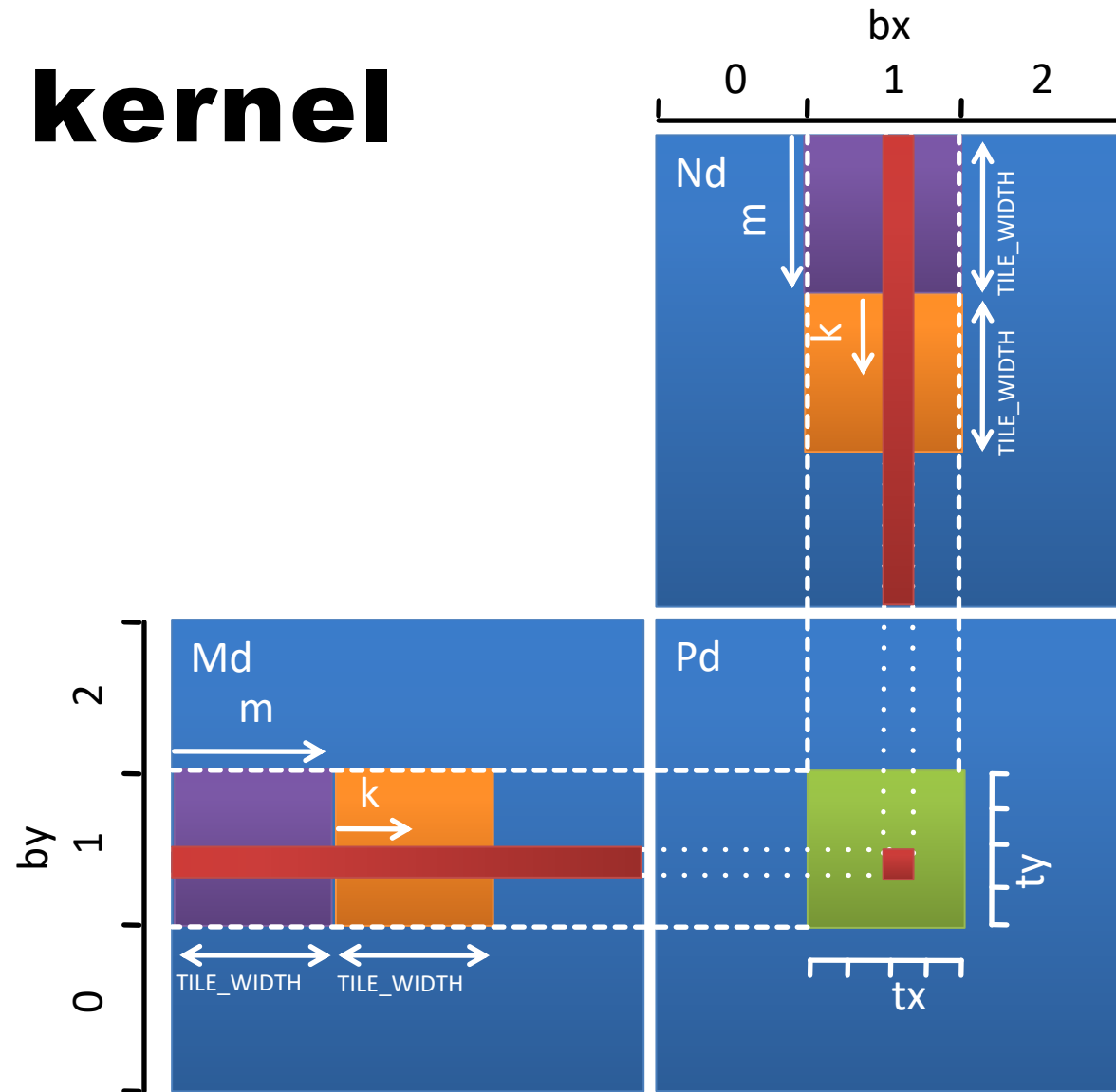

 Time

Plan for kernel



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming





```
__global__ void MatrixMulKernel(float* Md, float* Nd, float* Pd, int Width)
{
    __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
    __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x; int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    // Identify the row and column of the Pd element to work on
    int Row = by * TILE_WIDTH + ty;
    int Col = bx * TILE_WIDTH + tx;

    float Pvalue = 0;
    // Loop over Md and Nd tiles required to compute the Pd element
    for (int m = 0; m < Width/TILE_WIDTH; ++m) {

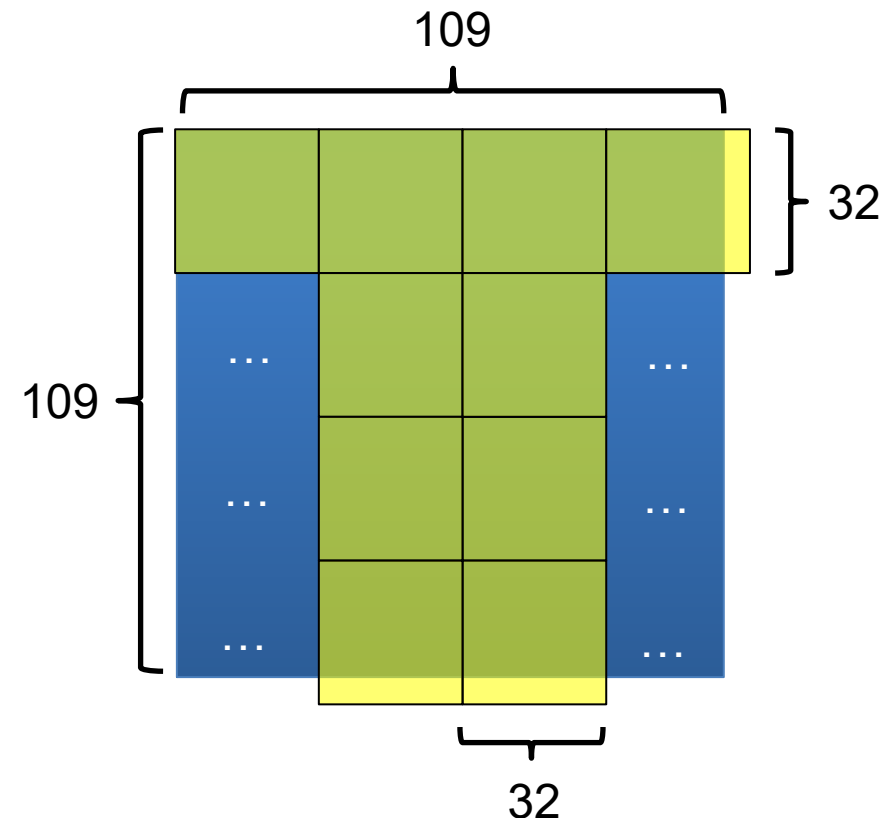
        // Collaboratively loading of Md and Nd tiles into shared memory
        Mds[ty][tx] = Md[Row * Width + (m*TILE_WIDTH + tx)];
        Nds[ty][tx] = Nd[(m*TILE_WIDTH + ty) * Width + Col];
        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; ++k)
            Pvalue += Mds[ty][k] * Nds[k][tx];
        __syncthreads();
    }
    Pd[Row * Width + Col] = Pvalue;
}
```

Tiled kernel

Boundary blocks

- What to do if the matrices are of size 109x109?
 - Too large for a single block (more than 1024 elements), cannot be divided into tiles without remainder
- Idea: Start enough blocks and handle the boundary in the kernel





Boundary handling in code

```
// ceil rounds to next larger integer
int blockDim = ceil(((double)Width)/((double)TILE_WIDTH));

dim3 blockDim(TILE_WIDTH, TILE_WIDTH);
dim3 gridDim(blockWidth, blockDim);

MatrixMulKernel<<<gridDim, blockDim>>>(Md, Nd, Pd, Width);
```

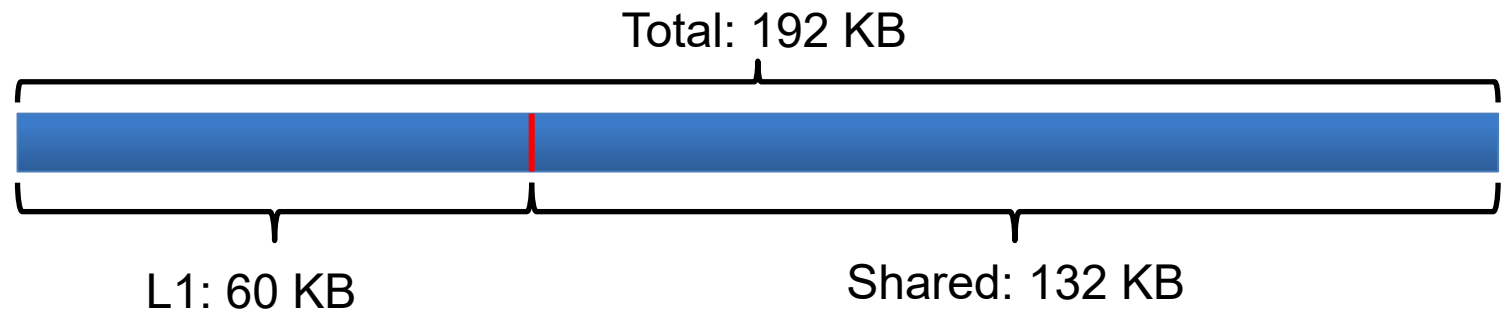
```
__global__
void MatrixMulKernel(float* Md, float* Nd, float* Pd, int Width) {
    [...]
    int Row = blockIdx.y * TILE_WIDTH + threadIdx.y;
    int Col = blockIdx.x * TILE_WIDTH + threadIdx.x;

    if (Row >= Width || Col >= Width) return;
    [...]
}
```



Size of shared memory

- Shared memory and L1 cache take up the same hardware
- Total size dependent on compute capability:
 - Turing (7.x): 96 KB
 - Volta (7.x): 128 KB
 - 8.0 and 8.7: 192 KB
 - 8.6 and 8.9: 128 KB
- Split can usually be configured per-kernel
 - For example: (0, 192); (8, 184); (16, 176); (32, 160); (64, 128); (100, 92); (132, 60); (164, 28);





Declaring shared-memory split

- Size of shared memory known at compile time?
 - No need for extra work
- Size of shared memory not known at compile time?
 - Set as a third parameter

```
int sharedMem = TILE_WIDTH * TILE_WIDTH * sizeof(float) * 2;  
MatrixMulKernel<<<gridDim, blockDim, sharedMem>>>(Md, Nd, Pd, Width);
```

- L1 cache is the rest
- Use `cudaDeviceSetCacheConfig()` and `cudaFuncSetCacheConfig()` to set preferences

Tiled matrix multiplication

- In general, if the matrix is of dimension N and the tile size is $TILE_WIDTH$, the dot product would be performed in $N/TILE_WIDTH$ phases
- With each phase focusing on a small subset of the input matrix values, the threads can
 - Collaboratively load the subset into shared memory
 - Use the values in the shared memory to satisfy their overlapping input needs in the phase
- Such focused behavior is called locality
- Note that Mds and Nds are reused to hold the input values



Memory as a limiting factor

- The more memory locations each thread requires, the fewer the number of threads that can reside in a multiprocessor
 - Registers
 - Shared memory
- Decision made at block granularity
 - Increasing the memory by a small amount may significantly reduce the number of threads

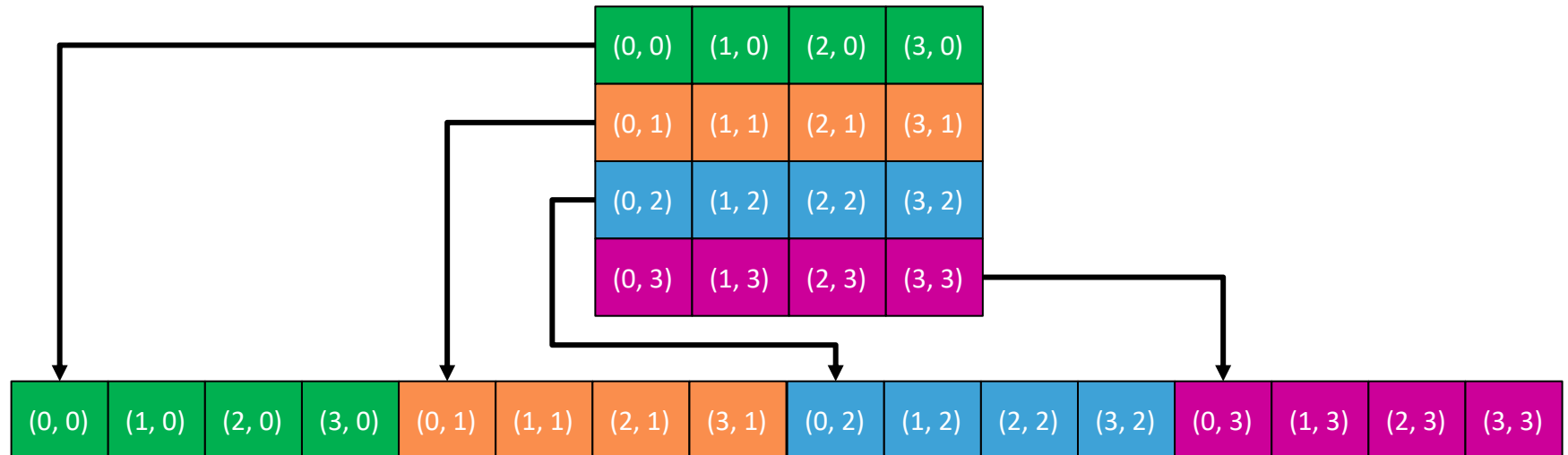


Summary CUDA memories

- Registers, shared memory, and constant memory can be accessed at higher speed than global memory
- Using these memories effectively requires careful algorithm design to enhance locality
- Programmers must be aware of limited memory sizes
 - Otherwise, the available parallelism diminishes
- Tiled algorithm also useful for other architectures that benefit from locality

Mapping thread blocks onto warps

- Thread blocks are partitioned into warps based on thread indices
 - Multidimensional blocks are linearized



- If the number of threads is not divisible by the warp size, the last warp is padded with extra threads
 - Execute instructions (like warp divergence), but results are nullified

Divergence

- Performance degradation if not all threads of a warp execute the same sequence of instructions
- May arise
 - At if-then-else statements where condition depends on thread identity
 - In for loops that iterate a different number of times for different threads
- All instructions executed by each thread; some are nullified
- Since Volta, threads can now diverge and reconverge at sub-warp granularity

Reduction algorithm

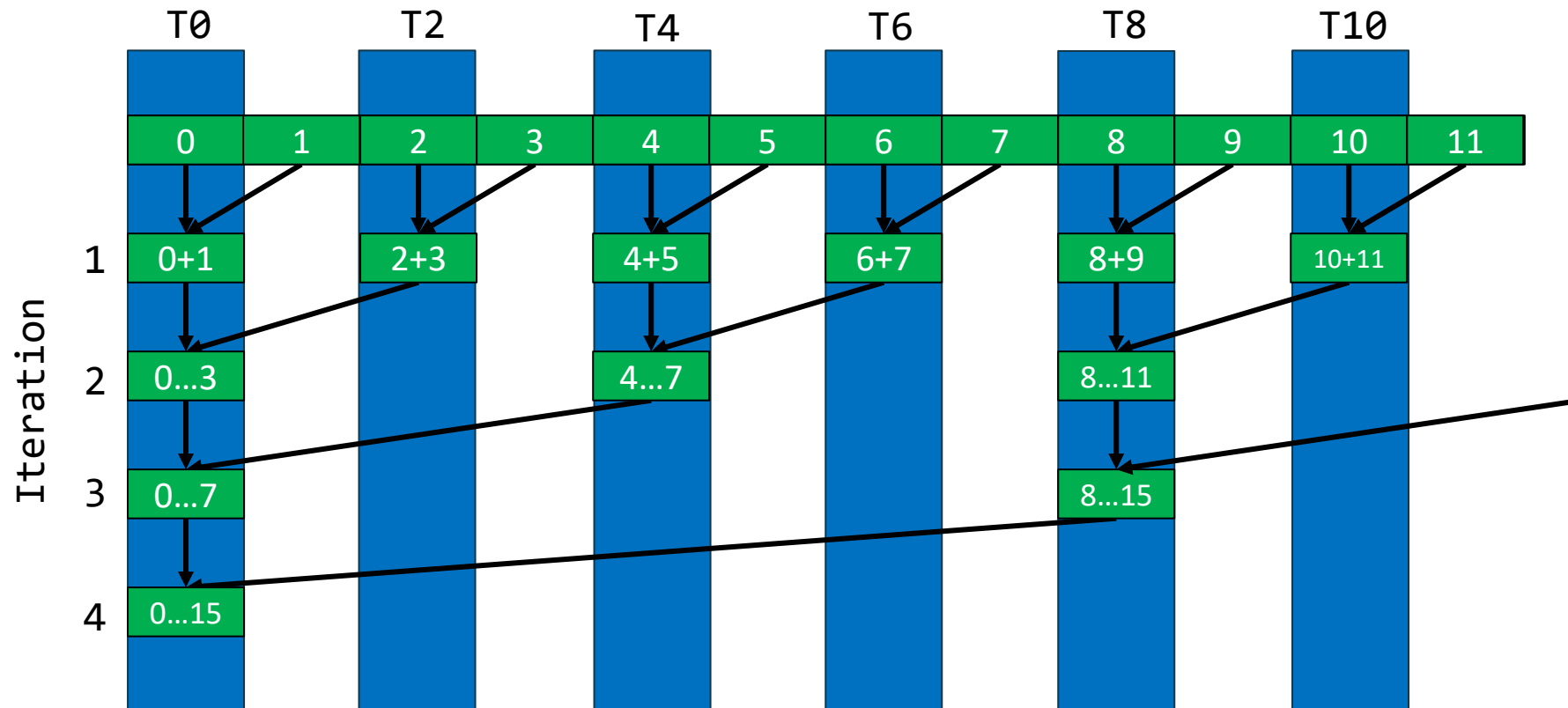
- Extracts a single value from an array of values
 - Example: sum, minimum, maximum
- Can be easily achieved by walking sequentially through the array
- Large arrays motivate parallel execution
- Idea: tournament reduction (“divide and conquer”)

```
__global__ void partialSum(/* parameters */) {  
    int t = threadIdx.x;  
    for (unsigned int stride = 1; stride < blockDim.x; stride *= 2) {  
        if (t % (2 * stride) == 0)  
            array[t] += array[t + stride];  
        __syncthreads();  
    }  
}
```

Execution of the sum reduction kernel

Parallel
Programming

- In each iteration, threads are divided into two groups with diverging behavior
- Requires two passes per iteration

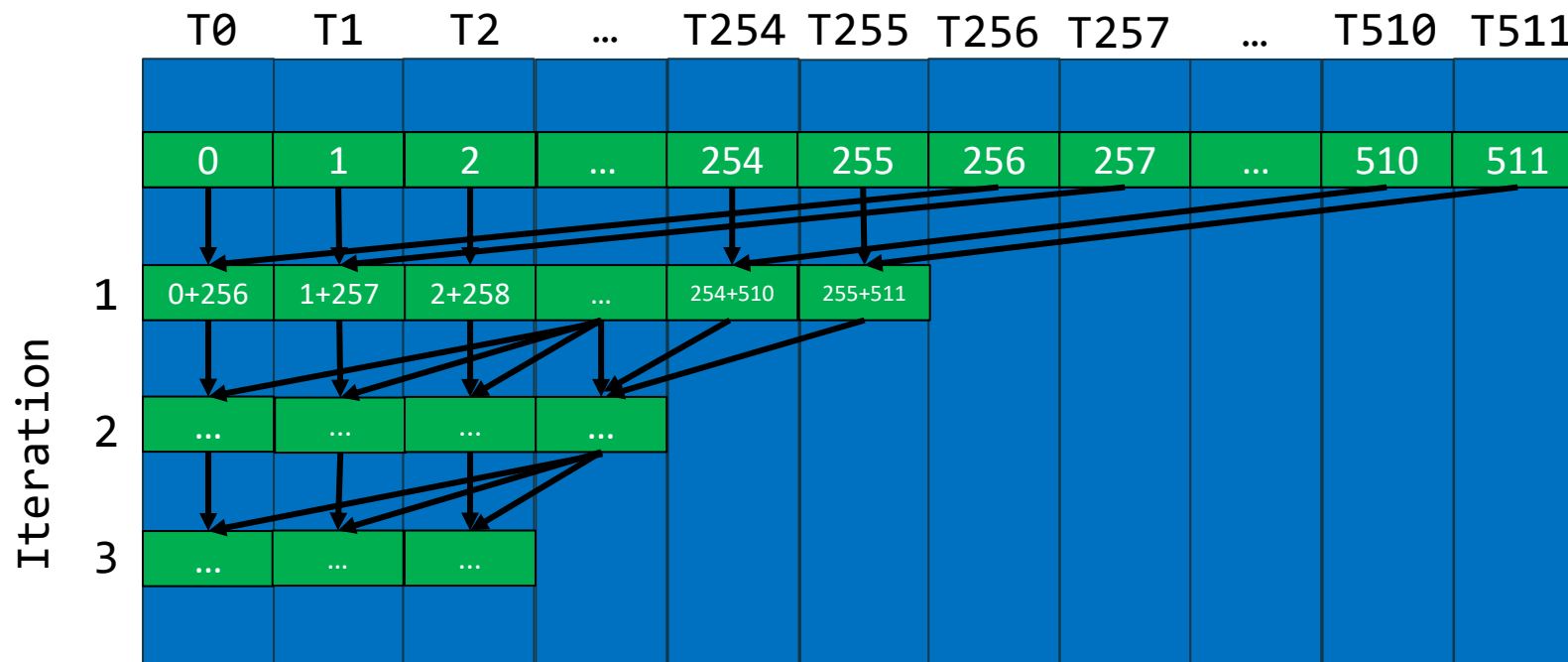


Revised kernel



```
__global__ void partialSum(/* parameters */) {  
    int t = threadIdx.x;  
    for (unsigned int stride = blockDim.x >> 1;  
         stride > 0; stride >> 1) {  
  
        __syncthreads();  
        if (t < stride)  
            array[t] += array[t + stride];  
    }  
}
```

Execution of revised sum reduction kernel



- Now divergence no longer within warps – except for last iterations

Further performance aspects (not covered in this course)

- (How to map thread blocks onto warps to avoid divergence)
- How to improve global memory bandwidth by coalescing memory accesses
- How to prevent bank conflicts
- (How to hide memory latency by prefetching data)
- How to optimize concurrency by balancing the per-thread resource requirements
- How to reduce cost of loop control by unrolling loops
 - Typically accomplished by compiler
- How to avoid redundant work by adjusting thread granularity

Is CUDA worth it?

It depends on the problem

Efficient CUDA codes require:

- Huge parallelism
- Little control flow
- Efficient data structures
- Enough calculation effort to make memory copies worthwhile



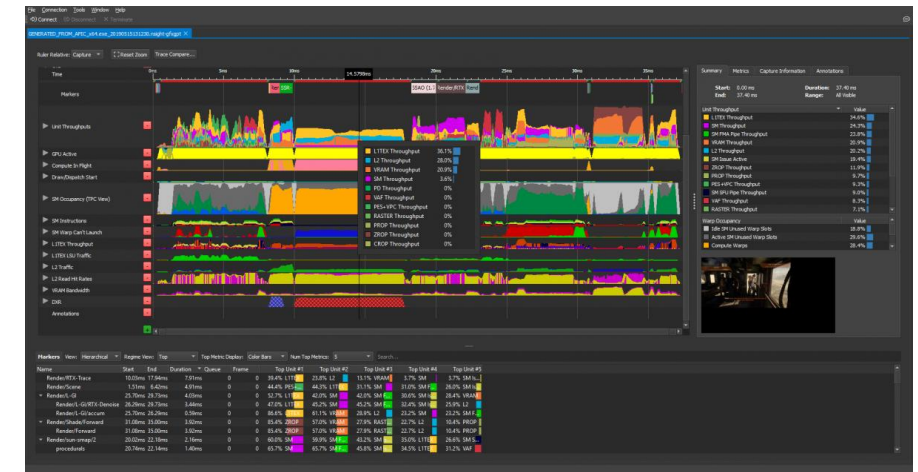
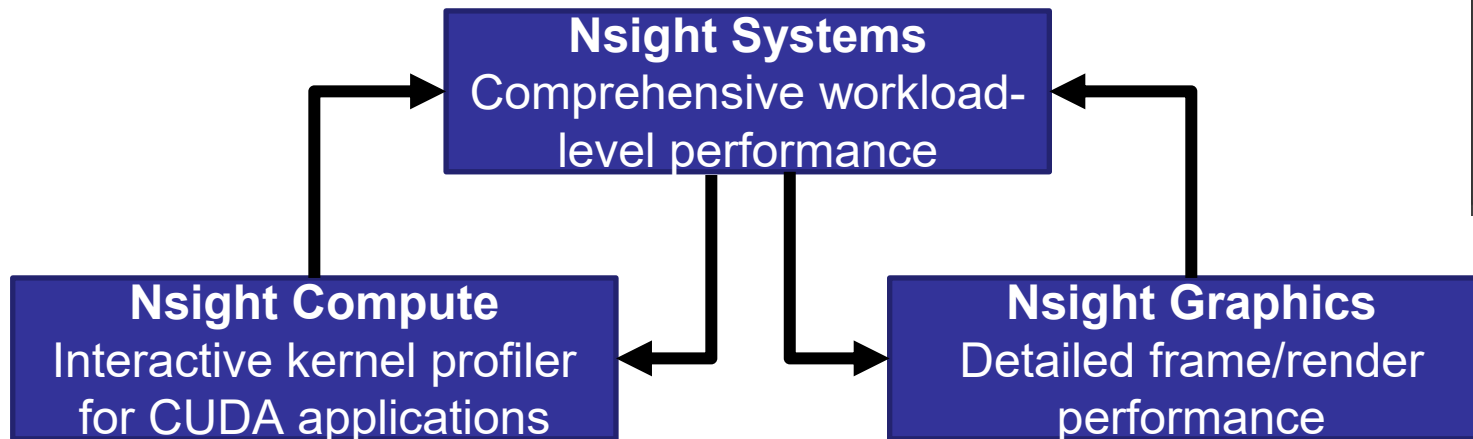
Advanced CUDA features

- NVIDIA Nsight
- Using multiple GPUs
- GPUDirect
- Dynamic parallelism
- Unified memory
- Cooperative groups
- Graphs (since CUDA 10)
- Tensor cores

Nvidia Nsight

Holistic system-level performance view

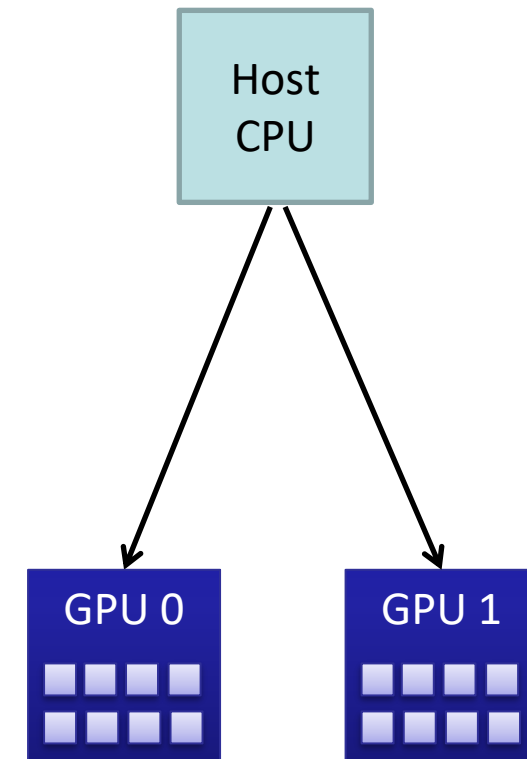
- Collects CPU, GPU and driver/kernel information
- Useful for profiling and optimizing bottlenecks





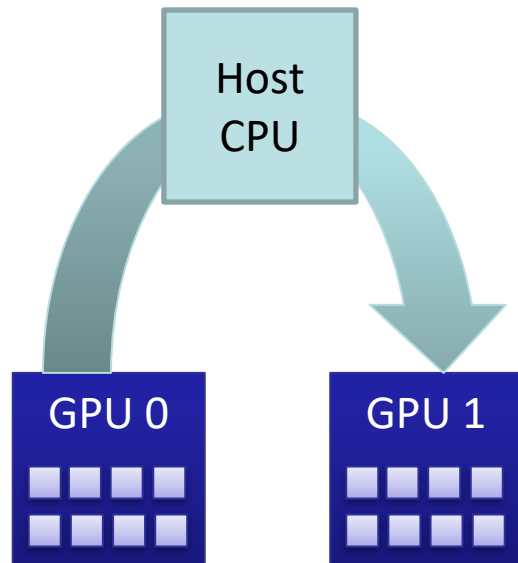
Using multiple GPUs

- Single host thread can access all the available GPUs
- Advantages
 - A large problem can be split across available GPUs
 - Easy to program
 - Direct data transfer from one device to another possible via GPUDirect technology

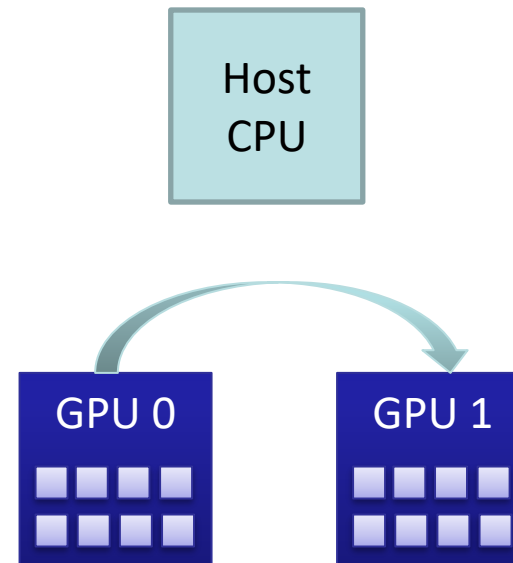


GPUDirect

- Allows direct communication from one device to another without involving the CPU



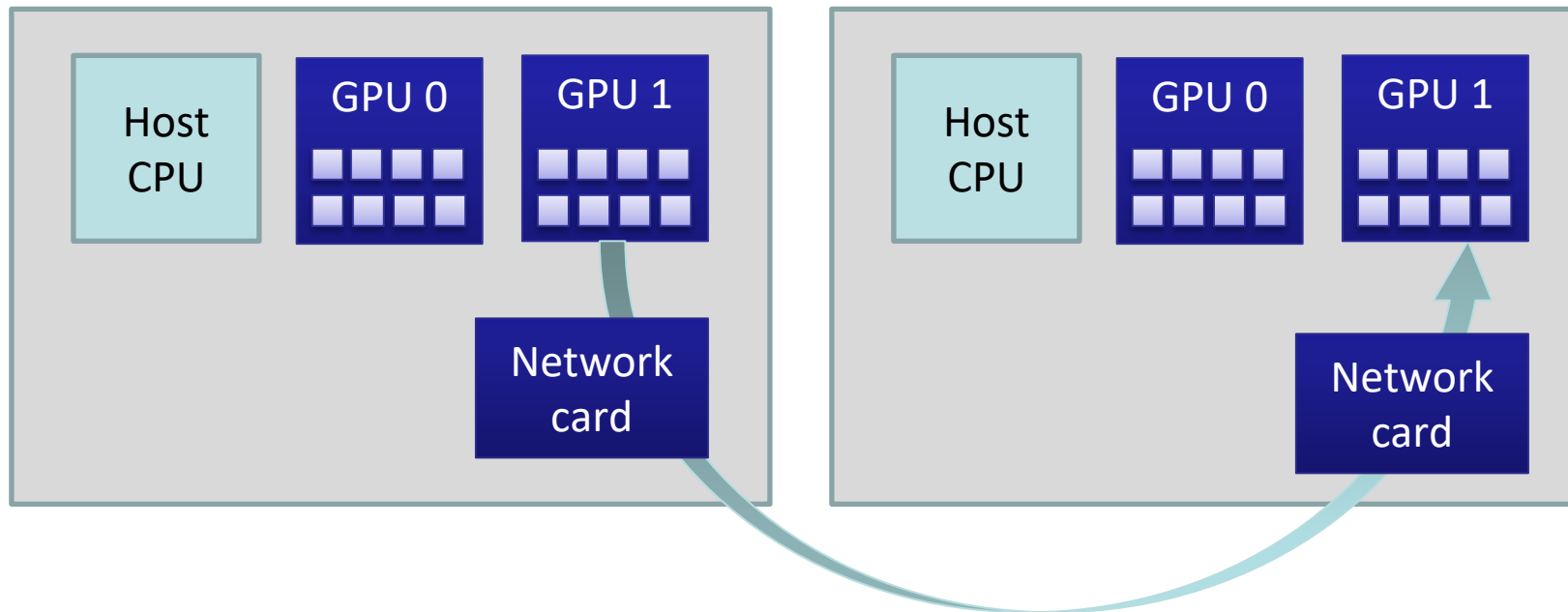
No GPUDirect



With GPUDirect

Remote direct memory access in GPUDirect

Enables direct communication between GPUs across the network

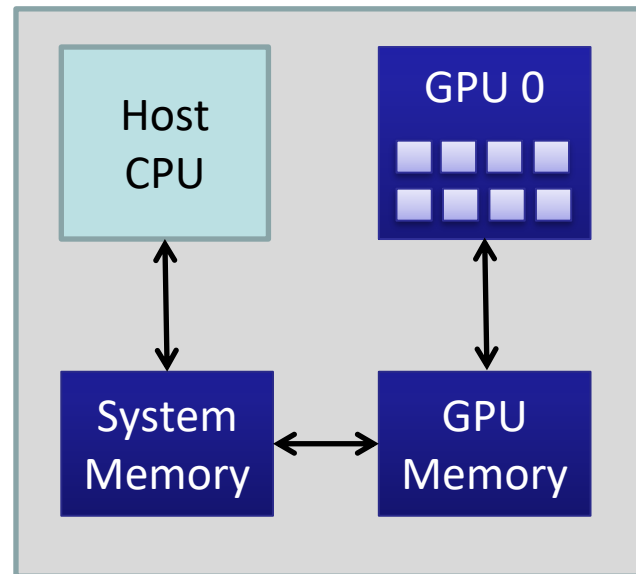


Dynamic parallelism

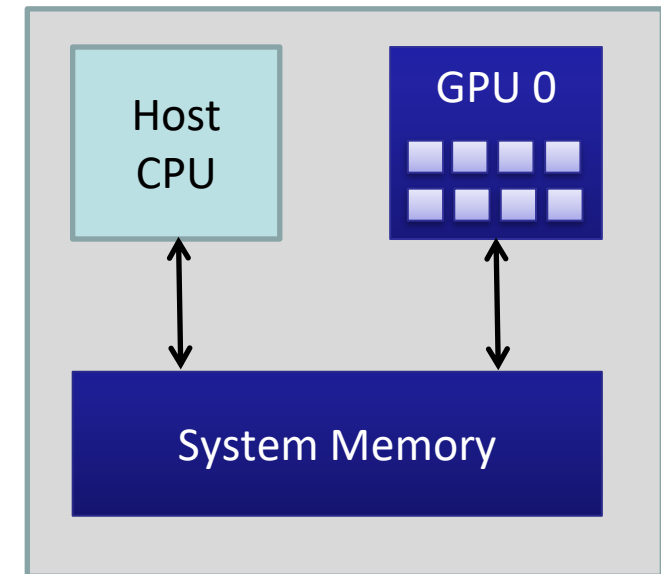
- A kernel can launch another kernel
- Supports recursive and data-dependent execution patterns including nested loops with differing amounts of parallelism
 - Example: adaptive grid refinement

Unified memory

- Avoids explicit copying of data
- CUDA runtime handles efficient placing and copying of data in memory transparently



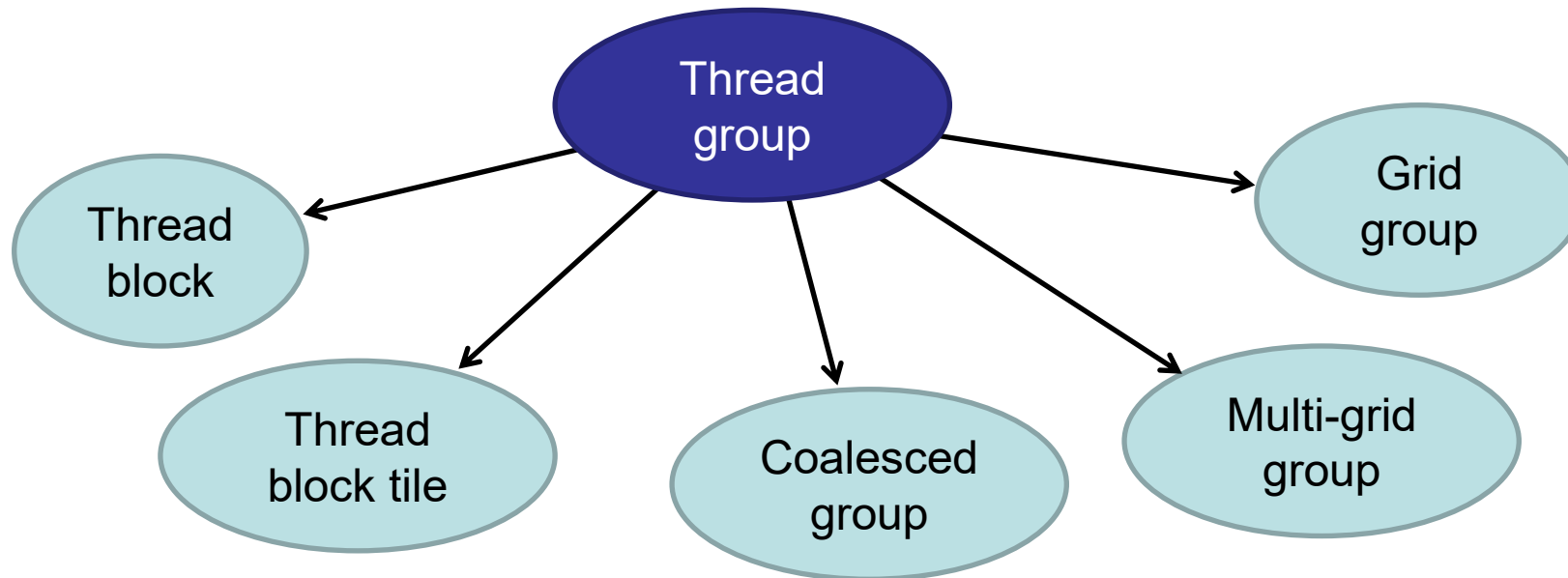
Developer view without
unified memory



Developer view with unified
memory

Cooperative groups

- Defining thread groups explicitly at sub-block and multi-block granularities
- Performing collective operations such as synchronization on them



CUDA Graphs

Source: <https://devblogs.nvidia.com/cuda-10-features-revealed/>



TECHNISCHE
UNIVERSITÄT
DARMSTADT

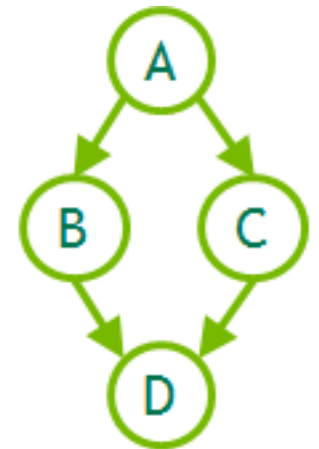
Parallel
Programming

- Scientific applications often require multiple GPU operations per iteration
- Graphs allow multiple GPU operations to be launched through a single CPU operation, and hence reduce overheads

```
// Define graph of work + dependencies
cudaGraphCreate(&graph);
cudaGraphAddNode(graph, kernel_a, {}, ...);
cudaGraphAddNode(graph, kernel_b, { kernel_a }, ...);
cudaGraphAddNode(graph, kernel_c, { kernel_a }, ...);
cudaGraphAddNode(graph, kernel_d, { kernel_b, kernel_c }, ...);

// Instantiate graph and apply optimizations
cudaGraphInstantiate(&instance, graph);

// Launch executable graph 100 times
for (int i=0; i<100; i++)
    cudaGraphLaunch(instance, stream);
```



Workflow Graph

Tensor cores

Specialized hardware execution units

- Perform matrix and convolution operations at a time (e.g., DL workloads)

Compared to scalar FP32 operations, tensor cores are:

- 8–16x faster (up to 32x faster with sparsity) and more energy efficient

$$\mathbf{D} = \begin{pmatrix} \begin{matrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{matrix} & \begin{matrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{matrix} \\ \text{FP16 or FP32} & \text{FP16} \end{pmatrix} + \begin{pmatrix} \begin{matrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{matrix} \\ \text{FP16 or FP32} \end{pmatrix}$$

Tensor core 4x4x4 matrix-multiply and accumulation (on Volta GPUs)



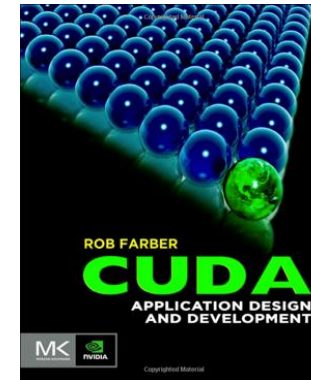
Summary CUDA

- GPU – multiprocessor of multiprocessors
 - Individual multiprocessors are SIMT architectures
 - Scheduling unit warp
- CUDA execution model – execution of a kernel as a grid of thread blocks
- Productivity advantage
 - Programmer writes code for single thread
 - Automatic thread management
- CUDA device memories
 - Registers, shared memory, global memory / constant memory



Further reading

- NVIDIA homepage <http://www.nvidia.com/>
- Rob Faber: CUDA Application Design and Development, Morgan Kaufmann, 2011
- Many books since then with varying focus
- Active areas of research
 - Higher-level programming abstractions that are automatically mapped onto CUDA
 - Autotuning of CUDA codes onto hardware platforms



Appendix - Hardware comparison



TECHNISCHE
UNIVERSITÄT
DARMSTADT

 Parallel
 Programming

Card	Tesla V100	Tesla A100	H100 SXM5
Architecture	Volta	Ampere	Hopper
SMs	80	108	132
FP32 CUDA cores per SM / per GPU	64 / 5120	64 / 6912	128 / 16896
FP64 CUDA cores per SM / per GPU	32 / 2560	32 / 3456	64 / 8448
Tensor cores per SM / per GPU	8 / 640	4 / 432	4 / 528
GPU boost clock	1530 MHz	1530 MHz	1830 MHz
Peak FP32 / FP64 / Tensor TFLOPs	15.7 / 7.8 / 125	19.5 / 9.7 / 312	66.9 / 33.5 / 989.4
Memory size	16 GB	40 GB	80 GB
L2 cache size	6144 KB	40960 KB	50 MB
Register file size per SM / per GPU	256 / 20480 KB	256 / 27648 KB	256 / 33792 KB
Max registers / thread	255	255	255
Shared memory size / SM	Up to 96 KB	Up to 164 KB	Up to 228 KB
Maximum shared memory / thread block	96 KB	163 KB	228 KB
Max thread blocks / SM	32	32	32
Warp size	32	32	32
Transistors	21.1 billion	54.2 billion	80 billion

Source: NVIDIA
Architecture Whitepapers

Appendix - Ampere GA100 Streaming Multiprocessor (SM)



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming



Appendix - Hopper GH100 Streaming Multiprocessor (SM)



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

